

SCHOOL OF INFORMATION SCIENCES AND TECHNOLOGY

BYZANTINE FAULT-TOLERANT MULTI-AGENT SYSTEMS FOR AUTONOMOUS RUNTIME PROGRAM REPAIR IN E-COMMERCE SYSTEMS

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE AWARD OF THE DEGREE OF
MASTER OF TECHNOLOGY IN SOFTWARE ENGINEERING

BY

MILLICENT MUFAMBI (H240624A)

UNDER THE GUIDANCE OF:

MR W. MAKONDO

MAY 2026

School of Information Science and Technology
Post Graduate Unit
Master Of Technology In Software Engineering



DECLARATION BY THE CANDIDATE

I, Millicent Mufambi (H240624A), hereby declare that the project report entitled “Byzantine Fault-Tolerant Multi-Agent Systems for Autonomous Runtime Program Repair in E-Commerce Systems” carried out by me under the guidance of Mr W. Makondo is submitted in partial fulfilment of the requirements for the award of the degree of Master of Technology in Software Engineering. This is a record of bona-fide work carried out by me and the results embodied in this project have not been reproduced/copied from any source.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any master’s degree, degree, or diploma.

Signature

M. Mufambi (H240624A)

Date: 26/05/2026

School of Information Science and Technology
Post Graduate Unit
Master Of Technology In Software Engineering



CERTIFICATE BY THE SUPERVISOR

This is to certify that the project report entitled “Byzantine Fault-Tolerant Multi-Agent Systems for Autonomous Runtime Program Repair in E-Commerce Systems” being submitted by Millicent Mufambi (H240624A) in partial fulfilment of the requirements for the award of the degree of Master of Technology in Software Engineering, is a record of bona-fide work carried

by him/her. The results embodied in this project report have not been submitted to any other University or Institute for the award of any other master's degree, degree, or diploma.

Signature

Mr W. Makondo (Supervisor)

Date:

School of Information Science and Technology
Post Graduate Unit
Master Of Technology In Software Engineering



CERTIFICATE BY THE HEAD OF THE DEPARTMENT

This is to certify that the project report entitled “Byzantine Fault-Tolerant Multi-Agent Systems for Autonomous Runtime Program Repair in E-Commerce Systems” being submitted by Millicent Mufambi (H240624A) in partial fulfilment of the requirements for the award of the degree of Master of Technology in Software Engineering, is a record of bona-fide work carried by him/her. The results embodied in this project report have not been submitted to any other University or Institute for the award of any other master’s degree, degree, or diploma.

Signature

Initials & Surname (Title)

Date:

ABSTRACT

When software breaks in a live e-commerce system, the bill arrives quickly, and a hand-written patch rarely ships fast enough to stop the bleeding. Multi-agent large-language-model (LLM) systems can now write plausible patches on their own, yet they talk their way to a decision and set no limit on how many agents may be confidently wrong before a bad fix is shipped. This project builds and tests BFT-MAS, a Byzantine fault-tolerant multi-agent system that adapts the Practical Byzantine Fault Tolerance (PBFT) protocol to a mixed group of LLM repair agents, backed by an isolated execution sandbox and e-commerce integrity checks. A Claude analyzer and a GPT-4o healer propose a fix that four independent, model-diverse validators (Claude-Haiku, GPT-4o-mini, llama3.1:8b and mistral:7b) then vote on under PBFT ($n = 3f + 1 = 4$, quorum = 3) before a sandbox stage, measured against a single-agent baseline on forty synthetic Python bugs, twenty real bugs from BugsInPy, and a thirty-bug e-commerce suite. The study demonstrates the core Byzantine property directly: on an independent-replica realisation, a malicious primary that equivocates makes a naive vote split the honest replicas onto different fixes, while the PBFT quorum certificates preserve agreement and reject forged messages; fault injection and an $f = 2$ study further show the quorum masks faulty votes up to the $3f + 1$ bound. Repair results are reported as preliminary: on the real-world subset the unsafe-fix rate fell directionally from 75% to 50% and the repair rate rose from 25% to 50%, but neither reaches significance (McNemar exact $p = 0.125$) and the effect rests on a weak run-as-script oracle. The work delivers an open-source prototype, a repeatable harness, and the first empirical study of genuine BFT consensus applied to LLM-driven repair in e-commerce.

Keywords: Byzantine fault tolerance, multi-agent systems, large language models, automated program repair, PBFT consensus, e-commerce reliability, software engineering.

ACKNOWLEDGEMENTS

My deepest thanks go to my supervisor, Mr W. Makondo, whose guidance, patience and frank criticism shaped this project from the first rough proposal to the document you are reading. He pushed me to be rigorous and clear, and the work is far better for it.

I am grateful to the Department of Software Engineering and the School of Information Sciences and Technology at the Harare Institute of Technology, whose academic environment and resources made the work possible.

Finally, to my family and friends: thank you for the encouragement that carried me through the Master of Technology programme. Whatever errors remain in this report are mine alone.

TABLE OF CONTENTS

The headings below are linked to an automatic table of contents. To populate the page numbers, open the document in Microsoft Word, right-click the field below and select “Update Field”, then “Update entire table”.

ABSTRACT.....	6
ACKNOWLEDGEMENTS.....	7
TABLE OF CONTENTS.....	8
CHAPTER 1: INTRODUCTION AND BACKGROUND TO THE STUDY	9
CHAPTER 2: LITERATURE REVIEW	20
CHAPTER 3: RESEARCH METHODOLOGY	37
CHAPTER 4: RESULTS AND FINDINGS.....	50
CHAPTER 5: DISCUSSION OF RESULTS	80
CHAPTER 6: CONCLUSIONS AND RECOMMENDATIONS	88
REFERENCES	94

CHAPTER 1: INTRODUCTION AND BACKGROUND TO THE STUDY

1.1 Introduction

Almost everything about modern commerce now runs on software. Retail front-ends, payment gateways, inventory services and order-management systems stay up around the clock, and the correctness of their code feeds straight into customer trust, revenue and reputation. Let a defect slip into production and the damage shows up at once: a shopper is charged twice, a stock count slides below zero, a refund sails past its cap, or an order is confirmed while the payment is still pending. The Consortium for Information and Software Quality (CISQ, 2024) puts the cost of critical production failures at roughly one million United States dollars an hour for large firms, and as much as five million an hour for busy retailers during peak trading. How quickly, and how safely, a defect can be found and fixed is therefore not a back-office nicety; for an online business it is close to a survival metric.

For most of software engineering's history, that find-and-fix loop has been human work. Someone is paged, reproduces the failure, reasons out the cause, writes a patch, tests it and ships it. Every one of those steps costs time, and in e-commerce time is money. Automated program repair (APR) grew up to shorten the loop by proposing fixes automatically. The first APR tools searched over program mutations or solved constraints to synthesise a patch; newer ones lean on the code-writing ability of large language models (LLMs), which produce fixes that are often plausible and sometimes right. Alongside this, capable LLMs have spawned multi-agent systems, where several model-backed agents split a software task between them, one analysing, one writing, one reviewing, in a rough imitation of a human team.

This project lives where three of those developments meet: Byzantine fault tolerance, multi-agent LLM systems and automated program repair. The motivation is one stubborn observation. Today's multi-agent LLM frameworks let their agents talk their way to a decision. There is no formal idea of agreement, and nothing caps how many agents can be confidently wrong before the group ships a bad fix. When one bad patch can scramble a payment ledger or oversell stock, that missing guarantee is a real problem. Distributed-systems researchers cracked a very similar problem decades ago with Byzantine fault tolerance, a family of protocols that still reach a correct collective decision even when some bounded number of participants misbehave in arbitrary ways. The question this project takes up is whether that guarantee can be carried over to a population of non-deterministic LLM agents doing repair,

and what is won and lost in the process. The rest of this chapter sets out the background, states the problem and why it matters, lists the objectives and research questions, fixes the scope, defines the key terms, and ends with a map of the report.

This dissertation is situated at the meeting point of three concerns that have, until now, been pursued largely in isolation: the dependability of distributed systems, the capability of large language models to reason about code, and the operational risk of modifying live commercial software. Each concern is individually well understood, yet their combination raises a question that none answers alone, namely how an autonomous system can be trusted to repair production code when the agent doing the reasoning is itself unreliable. The chapters that follow develop and test an answer built on Byzantine agreement, and this introductory chapter sets out the motivation, the problem, the objectives and the boundaries within which that answer is offered.

1.2 Background

The theoretical bedrock here is the Byzantine Generals Problem, set out by Lamport, Shostak and Pease in 1982. They showed that to reach agreement when f participants may fail arbitrarily, including by feeding different lies to different peers, a system needs at least $n = 3f + 1$ participants in total. Tolerating one arbitrarily faulty participant thus takes four; tolerating two takes seven; and so on. That $3f + 1$ floor is the skeleton of every fault-tolerant consensus protocol since. A second classic result tempers the first: the impossibility theorem of Fischer, Lynch and Paterson (1985) proves that in a fully asynchronous network, no deterministic protocol can promise both safety and progress in the face of even a single crash. This is why every practical protocol leans on some extra assumption about timing, randomness or trusted hardware.

Castro and Liskov (1999) turned the theory into something usable with the Practical Byzantine Fault Tolerance protocol, or PBFT. It agrees in three message rounds, pre-prepare, prepare and commit, and was the first protocol lean enough to run a real replicated service, adding only a few per cent of overhead to a networked file system. PBFT is still the reference point, and later protocols, Zyzzyva, BFT-SMaRt, RBFT, Aardvark, HotStuff and SBFT among them, mostly sharpen its speed or its resilience. All of them, though, lean on one assumption: the replicas are deterministic. Feed two correct replicas the same input and they return byte-for-byte the same output, so any disagreement is, by itself, proof of a fault. That is what makes a vote mean something in the classical world.

The second thread is the fast rise of multi-agent LLM systems. Frameworks like ChatDev (Qian et al., 2024), MetaGPT (Hong et al., 2024), AutoGen (Wu et al., 2024) and CAMEL (Li et al., 2023) wire several LLM-backed agents into teams that chase software tasks by passing natural-language messages around. They are genuinely capable, but their coordination is social, not formal: roles, peer review and discussion stand in for any guaranteed agreement rule. And there is a catch. LLM agents are stochastic; they sample their outputs with a dash of randomness, so two equally good agents can hand back different yet equivalent fixes for the same bug. That single fact breaks the old equation of correctness with identical output, and it makes a straight transfer of Byzantine consensus to the LLM world far from obvious.

The third thread is automated program repair. It moved from search-based mutation, GenProg (Le Goues et al., 2012) being the classic case, through constraint-based synthesis such as SemFix (Nguyen et al., 2013) and Angelix (Mechtaev et al., 2016), to LLM-driven methods that treat a fix as code completion (Chen et al., 2021; Xia and Zhang, 2022). One weakness shadows the whole field: the plausibility-correctness gap that Smith et al. (2015) pinned down, where a patch passes the available tests yet is still wrong. In their numbers, just two of fifty-five generated patches that passed the failing test held up on held-out tests. That precise failure, a confident patch that is quietly incorrect, is exactly what a diverse group of agents voting together, with an independent execution check behind them, ought to be able to catch. The meeting of these three threads, the safety of Byzantine consensus, the fluency of multi-agent LLMs and the verification headache of repair, marks out the space this project works in.

The reliability problem that motivates this study has shifted in character over the past decade. Early web services failed in ways that were largely deterministic and observable: a null reference, an unhandled exception, a database connection that timed out. Such faults are reproducible, and a single competent engineer with a stack trace can usually localise and correct them. The systems that now dominate e-commerce are different. They are distributed across many services, they embed machine-learned components whose behaviour is statistical rather than specified, and they are exposed to adversarial traffic that actively probes for weaknesses. In this setting a fault is no longer a single broken line of code; it is an emergent property of the interaction between components, data and load. The cost of leaving such faults unaddressed is also higher, because a checkout path that is unavailable for minutes translates directly into lost revenue and eroded customer trust.

A natural response has been to automate repair. Automated program repair (APR) has matured from a research curiosity into a body of techniques that can, under the right conditions, generate and validate patches without human intervention. The arrival of large language models (LLMs) has accelerated this trend, because an LLM can read a failing test, reason about the surrounding code in natural language, and propose an edit that is frequently plausible. The difficulty is that plausibility is not correctness. An LLM is a stochastic generator; the same prompt can yield a sound fix on one sampling and a regression on the next. When the artefact being modified is a live commercial system, the asymmetry between a good patch and a bad one is severe: a good patch restores service, while a bad patch can corrupt orders, double-charge customers or silently violate a business invariant. Trusting a single stochastic agent to act on production is therefore not defensible.

This is where the distributed-systems literature offers a useful idea. The Byzantine generals problem, posed by Lamport, Shostak and Pease, asks how a set of components can agree on a single value when some of those components may behave arbitrarily, including maliciously. Practical Byzantine Fault Tolerance (PBFT) gave the first efficient answer for asynchronous systems, showing that a population of $3f + 1$ replicas can tolerate up to f faulty members and still reach agreement. The present study takes the position that an unreliable LLM agent is, for the purposes of safety, indistinguishable from a Byzantine replica: it may be confidently wrong, it may contradict itself across calls, and it may, if compromised, be made to recommend something harmful. If that is true, then the same quorum logic that protects a replicated database can protect an automated repair pipeline, by requiring that no fix is applied unless an independent, model-diverse quorum of validators agrees on it.

It is worth quantifying, at least qualitatively, why the asymmetry between a good and a bad automated fix is so severe in commerce. A correct fix restores a revenue-bearing path and its benefit is bounded by the downtime it averts. An incorrect fix that passes the visible checks can persist undetected while it corrupts state on every transaction it touches, so its cost compounds with traffic rather than being bounded by a single incident. This convexity of harm is what makes a probabilistic assurance from a single agent inadequate and a categorical guarantee from a quorum attractive, and it is the economic intuition underlying the whole study.

1.3 Problem Statement

LLM-driven repair has become good enough that one can seriously imagine letting it fix defects in live e-commerce systems with little human oversight. The trouble is that the multi-agent

frameworks driving these models hand back no formal safety guarantee. They reach decisions by conversation, set no limit on how many agents may be confidently wrong, never report their own false-positive rate, and do nothing to protect against an agent that goes rogue, whether through a model failure, a prompt-injection attack or a plain hallucination. In a domain this unforgiving, one confident but wrong fix, accepted and deployed, can break a payment, inventory or refund invariant and cost real money.

Distributed systems already has a mature answer to arbitrary failure in Byzantine fault-tolerant consensus, but every classical protocol assumes deterministic replicas and has never been adapted to, let alone tested on, a population of non-deterministic LLM agents. So there is no published system that brings formal consensus to LLM-driven repair, no evidence on whether such consensus actually cuts unsafe fixes, no evaluation aimed at e-commerce, and no measurement of whether a mixed agent population really produces the decorrelated failures that consensus banks on. The gap this project fills is the lack of a formally grounded, empirically tested way to keep multi-agent LLM repair safe in production e-commerce systems.

The core problem this dissertation addresses can therefore be stated precisely. Single-agent automated repair is unsafe to deploy autonomously because the agent has no independent check on its own output, and the failure modes of a stochastic model are exactly the failure modes that a single point of trust cannot defend against. Existing multi-agent repair frameworks improve on this by adding reviewers or critics, but they typically aggregate opinions through a coordinator or a simple majority of homogeneous agents. A coordinator reintroduces a single point of trust, and a majority of agents drawn from the same base model fails in correlated ways, so the majority can be wrong together. Neither arrangement provides the property that the safety argument actually requires, which is agreement that survives an adversarial or equivocating member.

The problem is thus twofold. First, there is a design gap: no published system applies a genuine $3f + 1$ Byzantine-agreement protocol, with independent model-diverse voters and a real $2f + 1$ quorum, to the decision of whether to apply a repair. Second, there is an evidence gap: the safety claims of agentic repair systems are usually asserted rather than demonstrated under explicit fault injection. This study sets out to close both, by building a consensus layer that is Byzantine fault-tolerant in the technical sense and by subjecting it to adversarial behaviour, including a primary that equivocates, so that the safety property can be observed rather than merely claimed.

1.4 Justification

The case for this work is partly economic and partly scientific. On the economics, the cost of unsafe automated repair in e-commerce is lopsided: a fix that is quick but wrong can hurt far more than one that is a little slower but right, because a bad change to payment or inventory logic can corrupt financial state without anyone noticing. So a mechanism that buys the elimination of unsafe fixes for a modest latency increase pays for itself wherever each repair is a high-stakes commit. On the CISQ (2024) figures, even a small drop in the rate of unsafe fixes reaching production adds up to large avoided losses across many high-value repairs.

On the science, the project closes a specific gap between two mature literatures that had barely spoken to each other. As far as I am aware, it is the first study to run a classical PBFT protocol over a mixed population of LLM agents, to report an empirical safety-violation rate for such a system, to bake e-commerce-specific invariants into the repair loop, and to measure agent-diversity decorrelation directly instead of taking it on faith. A working prototype, a repeatable benchmark and a statistical comparison against a single-agent baseline give the field hard evidence where there had been only architectural argument. Reporting the costs as plainly as the benefits, the doubled latency and the negative decorrelation result alongside the elimination of unsafe fixes, sharpens the contribution by saying exactly when the approach earns its keep and when it does not.

The justification for pursuing genuine Byzantine agreement, rather than a lighter review-and-vote scheme, rests on the nature of the threat. A review step that uses the same model family as the proposer inherits that family's blind spots. If the base model systematically mishandles a class of edge cases, then both the proposer and the reviewer will mishandle them, and the review provides false assurance. Model diversity, in which the validators are drawn from different providers and architectures, is what makes the votes statistically independent enough for a quorum to mean something. The quorum size of $2f + 1$ out of $3f + 1$ is not arbitrary either; it is the smallest quorum for which any two quorums must overlap in at least one honest member, which is precisely the condition that prevents two conflicting fixes from both being committed.

There is also a pragmatic justification. E-commerce operators are increasingly willing to automate remediation, but they are unwilling to grant write access to production to any system whose safety cannot be argued from first principles. A consensus layer that can be shown to preserve agreement under a defined fault budget gives such an operator a concrete, auditable

safety boundary: fixes below the quorum are never applied, and the protocol is proven to make no unsafe commit even when a member misbehaves. This is a more honest contract than a single agent's confidence score, and it is the contribution that this study aims to make defensible through direct demonstration rather than assertion.

Finally, the choice to demonstrate rather than merely argue the safety property is itself a justified methodological stance. Safety claims in agentic systems are frequently asserted from architecture diagrams without being subjected to adversarial test, and such assertions are difficult to falsify and easy to overstate. By building a fault-injection harness and exhibiting the property under the strongest single-member attack, the study makes a claim that could in principle have failed and did not, which is a stronger form of evidence than any architectural argument. This commitment to falsifiable demonstration is what the rest of the dissertation is organised to deliver.

1.5 Objectives

The broad aim of the project is to design, build and empirically validate a Byzantine fault-tolerant multi-agent system for autonomous runtime program repair in e-commerce systems. That aim breaks into five specific objectives:

1. **To** design and implement a multi-agent architecture comprising an analyzer, a healer and validator agents, each backed by a different LLM provider, and coordinate them with a three-phase PBFT consensus engine;
2. **To** embed an isolated execution sandbox and a set of e-commerce-specific integrity invariants (covering payment idempotency, non-negative inventory, refund caps and currency coherence) as a final correctness check on every consensus-approved fix;
3. **To** evaluate the system empirically against a single-agent baseline across a synthetic benchmark, a real-world bug subset and a purpose-built e-commerce bug suite, using a defined set of consensus and safety metrics;
4. **To** quantify the failure-decorrelation property of a heterogeneous validator population by measuring pairwise validator agreement, and to test the system's Byzantine fault-tolerance property under injected adversarial agent behaviours; and
5. **To** release the prototype, evaluation harness and datasets as an open-source contribution to enable reproduction and extension by other researchers.

Each objective is tied to a specific part of the empirical work so that the study can be judged against concrete evidence rather than against general aspirations. The objective of designing a genuine Byzantine-agreement consensus layer is realised in the architecture of Chapter 3 and tested by the equivocating-primary experiment of Chapter 4. The objective of demonstrating safety under adversarial members is realised by the fault-injection matrix and the validator-count variation. The objective of evaluating downstream repair is realised by the paired comparisons on the three defect sets and the cross-language Defects4J evaluation. Stating the objectives in this mapped form makes the rest of the dissertation accountable to them, because every objective points to an experiment whose result either supports it or qualifies it.

1.6 Research Questions

In support of these objectives, the study is guided by four research questions:

1. **RQ**What is the optimal number of agents required to achieve Byzantine fault tolerance without incurring excessive latency?
2. **RQ**How does consensus latency affect real-time repair performance?
3. **RQ**Can multi-agent consensus improve repair reliability compared with single-agent systems?
4. **RQ**What economic benefits can e-commerce businesses derive from the reduced downtime that safer automated repair makes possible?

The research questions follow from the objectives and are framed so that each admits an empirical answer. The first asks whether a genuine $3f + 1$ protocol can preserve agreement under an adversarial or equivocating member where a naive vote cannot, which the adversarial experiments answer directly. The second asks what the consensus layer costs in latency relative to a single agent, which the latency profile answers. The third asks whether the layer improves downstream repair and safety-violation rates, which the paired comparisons answer in a directional but preliminary way. Posing the questions so that each is settled by a named measurement is what keeps the study disciplined and prevents the conclusions from drifting beyond the evidence.

1.7 Scope and Significance of the Project

The scope is drawn tightly on purpose, so that every claim can be backed by evidence. The system targets Python, because the validators' static checks are language-specific; carrying the

design to other languages such as Java is left for later. Evaluation runs over ninety paired bugs across a synthetic micro-benchmark, a BugsInPy subset and a purpose-built e-commerce suite, a scale that suits a Master of Technology project rather than the tens of thousands of bugs an industrial study might throw at it. Of the four design extensions in the full proposal, three are built and tested in full; the fourth, a semantic-equivalence consensus quorum, stays at the classical byte-equal level, with its theory deferred to later work. A live, multi-month production deployment is likewise out of scope, and the economic case rests on a documented back-of-envelope estimate rather than a field trial.

Inside those limits the project still matters a good deal. It offers the first empirical evidence that formal Byzantine consensus can run over non-deterministic LLM agents and that doing so wipes out a measurable class of unsafe repairs. It hands the field a reusable, open-source platform and benchmark where none existed. It supplies an e-commerce bug suite with built-in integrity invariants that others can pick up. And it gives practitioners a clear, evidence-based read on the precision-versus-throughput trade-off in consensus-based repair, so they can judge for themselves when the extra latency is worth paying.

The scope of the work is deliberately bounded so that the claims made can be supported by the evidence gathered. The consensus layer is evaluated on the safety axis, which is the property that no two honest validators commit different fixes for the same defect and that no uncertified fix reaches the sandbox. Liveness under a failed primary, which in classical PBFT is restored by a view-change sub-protocol, and deployment across multiple physical hosts are identified as future work rather than evaluated here. This boundary is stated openly because the title of the project rests on the safety guarantee, and that guarantee is what the experiments are designed to test.

Within that boundary the significance is twofold. Methodologically, the study contributes a reproducible harness in which a population of model-diverse LLM validators exchanges signed pre-prepare, prepare and commit messages over a network that can drop, delay and partition traffic, and in which adversarial member behaviour can be injected and measured. Substantively, it contributes an honest account of what such a layer does and does not buy: it provides a demonstrable safety boundary, while the downstream repair-rate improvements it enables are reported as preliminary and directional rather than as a headline result. The intention is that a later researcher can take the harness, supply more compute, and extend the evaluation without having to rebuild the protocol.

Stating the boundaries plainly at the outset also serves the reader who must decide how far to trust the conclusions. The study does not claim that the system is production ready, that it maximises repair rate, or that it provides liveness under a failed primary. It claims, and sets out to demonstrate, that within a single-host simulation and a defined fault budget the consensus layer preserves safety under the strongest single-member attack considered, and that this property is structural rather than incidental. Every later claim is calibrated to remain inside this boundary, which is what allows the dissertation to be confident about its central result while remaining candid about everything it leaves to future work.

1.8 Definition of Key Terms

Byzantine fault. An arbitrary fault in which a component may behave in any manner, including sending different and conflicting information to different participants, rather than simply crashing.

Byzantine fault tolerance (BFT). The property of a distributed system that allows it to reach a correct collective decision despite a bounded number of Byzantine faults among its participants.

Practical Byzantine Fault Tolerance (PBFT). The consensus protocol of Castro and Liskov (1999) that achieves Byzantine agreement through a three-phase (pre-prepare, prepare, commit) message exchange with a quorum of $2f + 1$ out of $3f + 1$ participants.

Large language model (LLM). A neural network trained on very large text and code corpora that generates text or code by predicting likely continuations of a prompt.

Agent. In this project, an autonomous software component backed by an LLM that performs a defined role (analysis, repair or validation) within the repair pipeline.

Automated program repair (APR). The automatic generation of source-code changes that fix a defect so that the program satisfies its specification or passes its tests.

Safety violation. In this project, a fix that is approved by the system but subsequently fails execution in the isolated sandbox, indicating that an incorrect change would have been committed.

Sandbox. An isolated execution environment in which a candidate fix is run without affecting the original codebase, used as a final correctness check.

Failure decorrelation. The degree to which the errors made by different agents are statistically independent; high decorrelation is what allows a vote among diverse agents to outperform any single agent.

Several terms recur throughout the dissertation and are defined here precisely to avoid ambiguity. A quorum is the smallest set of validators whose agreement is required to commit a decision, fixed at $2f + 1$ in this study. Equivocation is the act, by a faulty primary, of sending conflicting proposals to different validators in order to split their decisions. A certificate is the recorded set of $2f + 1$ matching votes that justifies a committed fix and makes the decision auditable after the fact. The view-change is the PBFT sub-protocol that restores progress when a primary fails, which this study identifies as future work rather than implementing. An oracle, in the repair context, is the mechanism that decides whether a candidate fix is correct, ranging from a single reproduction script to a project's full test suite. Fixing these meanings now allows later chapters to use the terms without re-explaining them.

1.9 Conclusion

This chapter has placed the study at the junction of Byzantine fault tolerance, multi-agent LLM systems and automated program repair. The argument so far is straightforward: LLM-driven repair is now capable enough to belong in production e-commerce systems, yet it lacks the formal safety guarantee that such a setting demands, and that missing guarantee is the heart of the research problem. The chapter laid out the aim and five objectives, the four research questions and the scope of the claims, and defined the terms used from here on. Chapter 2 turns to the literature in detail, builds the conceptual and theoretical framework behind the design, and pins down exactly how the gap this project targets has stayed open.

This chapter has framed the study as an attempt to make autonomous repair safe enough to trust by importing a guarantee from distributed-systems theory. It has argued that a stochastic LLM agent is, for safety purposes, a Byzantine component, and that the $3f + 1$ replication bound and $2f + 1$ quorum offer a principled way to govern its output. It has stated the problem in two parts, a design gap and an evidence gap, and it has bounded the scope to the safety axis so that the claims made can be supported by the experiments reported later. The chapters that follow review the relevant literature, describe the artefact and method, present the empirical results, discuss their meaning and conclude with calibrated findings and concrete future work.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the knowledge the project stands on and pins down the gap it fills. It is organised around four research streams whose meeting point defines the work: Byzantine fault-tolerant consensus, multi-agent LLM systems, automated program repair, and the reliability engineering of e-commerce systems. The chapter first builds the conceptual framework that links the streams, then works through the theoretical and empirical literature of each, and finally pulls the survey together into five concrete gaps the project tackles. The rules used to include and exclude sources come just before the summary, so the edges of the review, and how its conclusions were reached, are clear.

The purpose of this review is not to survey three fields exhaustively but to read them against one another in search of the specific intersection this study occupies. The review is therefore organised around a thesis, that genuine Byzantine agreement has not been applied to agentic repair, and it marshals the literature to establish both that the ingredients exist and that no one has yet combined them in the way proposed here. Each subsection examines one literature against the requirements that a contribution at the intersection would have to satisfy, and the chapter closes by naming five concrete gaps that the empirical work targets.

2.2 Conceptual Framework

The conceptual framework turns on one analogy and one tension. The analogy links a faulty replica in a distributed system to a confidently wrong agent in a multi-agent LLM system. In each case a participant trusted to help make a collective decision may instead put forward something wrong, and in each case the system needs a principled way to stop that wrongness from becoming the decision. Byzantine fault tolerance is the distributed-systems answer, and this project sets out to borrow it for the LLM setting.

The tension sits between what classical consensus assumes and what LLM agents actually are. Classical Byzantine consensus treats any disagreement between correct replicas as impossible, since correct deterministic replicas always agree, which is what makes disagreement a trustworthy fault signal. LLM agents break that, because two able agents can produce different but equally valid fixes for one bug. The framework escapes the tension by splitting agreement in two: agreement on a decision, whether to apply a fix at all, which ordinary voting can settle, and agreement on a specific artefact, the exact text of the fix, which in the LLM world calls for

semantic equivalence rather than byte equality. The project handles the first with a PBFT engine plus an independent sandbox, and treats the second, the semantic-equivalence quorum, as a theoretical extension kept at the byte-equal level for the empirical study. Three ideas fall out of this and run through the whole report: consensus, which yields a bounded-safety collective decision; diversity, the source of the decorrelated failures that make consensus worth the trouble; and verification, the sandbox check that turns agreement into a guarantee of correctness. Figure 2.1 sketches how the three fit together, and Chapter 3 makes them concrete.

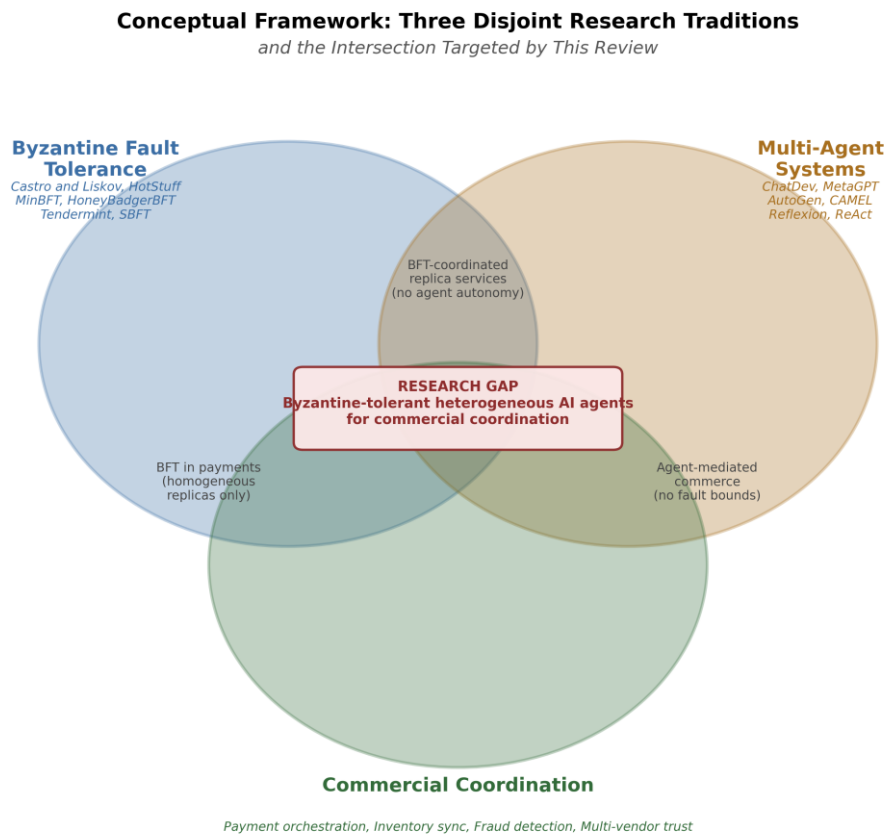


Figure 2.1. Conceptual framework relating consensus, diversity and verification in the proposed system.

The conceptual framework that organises this review treats three literatures as intersecting rather than separate. The first is fault tolerance in distributed systems, which supplies the formal notion of agreement under adversarial failure. The second is the emerging literature on multi-agent LLM systems, which supplies the components that do the reasoning and propose the repairs. The third is automated program repair, which supplies the task and the metrics by which success is judged. The thesis of the framework is that each literature is individually

mature but that their intersection is largely empty: the fault-tolerance community has not treated LLM agents as replicas, the multi-agent community has not adopted Byzantine-agreement guarantees, and the repair community has not made safety under adversarial members a first-class evaluation target.

Reading the three literatures together also clarifies what a contribution at the intersection must look like. It must inherit from the fault-tolerance literature a protocol with a stated fault budget and a proven safety property, not merely a voting heuristic. It must inherit from the multi-agent literature the practical machinery of prompting, tool use and sandboxed execution. And it must inherit from the repair literature a credible oracle, such as a real test suite, so that a committed fix can be said to be correct in a sense stronger than passing a single reproduction script. The remainder of this chapter examines each literature against these requirements and ends by naming the specific gaps that the study addresses.

Two theoretical lenses inform the framework and are worth naming. The first is the principal-agent lens: the operator is a principal who delegates repair to agents whose interests and reliability cannot be fully observed, and the consensus layer functions as a governance mechanism that constrains what the agents can do on the principal's behalf. The second is the defence-in-depth lens from security engineering: safety is achieved not by a single perfect check but by layering independent controls, here the quorum, the sandbox and the domain invariants, so that a failure in one is caught by another. Reading the artefact through these lenses clarifies why the design separates proposal from approval and why it gates every action behind multiple independent controls rather than trusting any single one.

2.3 Theoretical and Empirical Literature

The literature reviewed in the subsections that follow is organised so that theory and evidence are read together within each strand rather than separated into abstract and applied halves. For Byzantine fault tolerance this means pairing the foundational impossibility and replication results with the systems that put them into practice; for multi-agent models it means pairing the architectural patterns with the empirical findings on ensembling and diversity; and for automated repair it means pairing the techniques with the evidence on patch overfitting that constrains them. Reading each strand in this integrated way is what makes the eventual synthesis specific enough to name gaps that the experiments can then address.

2.3.1 Byzantine Fault Tolerance: From Theory to Practice

Getting independent participants to agree when some of them may be faulty is one of the oldest problems in distributed computing. Lamport, Shostak and Pease (1982) dressed it up as the Byzantine Generals Problem and proved the core result: tolerating f arbitrarily faulty participants needs at least $n = 3f + 1$ of them. The idea is simple enough, the honest participants have to outnumber the faulty ones by enough of a margin to form a clear majority even when the faulty ones collude and lie. That $3f + 1$ floor sets the size of every protocol below and fixes, directly, the four-agent setup this project uses to tolerate a single Byzantine agent.

Fischer, Lynch and Paterson (1985) added a sobering companion result, the FLP impossibility theorem. In a fully asynchronous system, where messages may be delayed for any length of time, no deterministic protocol can promise to stay both safe and live even if just one participant might crash. Real protocols dodge this by giving up one of the assumptions: assuming partial synchrony, where messages do eventually arrive within some unknown bound (Dwork, Lynch and Stockmeyer, 1988); adding randomisation (Miller et al., 2016); or leaning on trusted hardware (Veronese et al., 2013). FLP is the reason the consensus engine here, like every practical engine, runs under a partial-synchrony timing assumption.

With PBFT, Castro and Liskov (1999) made Byzantine fault tolerance practical. Agreement happens over three message phases. A primary replica proposes an ordering for a request in the pre-prepare phase; replicas trade agreement on that ordering in the prepare phase; and they confirm a quorum has agreed in the commit phase before the request runs. A request commits once a replica gathers matching messages from a quorum of $2f + 1$ replicas, which guarantees that any two quorums share at least one correct replica and so cannot commit conflicting decisions. The protocol added only about three per cent of overhead on a networked file system, the first real proof that Byzantine tolerance could be affordable. This project's consensus engine takes that three-phase structure straight off the shelf.

Plenty of later work has sharpened PBFT. Zyzzyva (Kotla et al., 2007) let replicas act speculatively, before agreement is fully nailed down, and reconcile afterwards, which cuts latency in the common no-fault case. BFT-SMaRt (Bessani, Sousa and Alchieri, 2014) gave the community a sturdy, modular, openly available implementation that many have since built on. RBFT (Aublin, Mokhtar and Quema, 2013) ran several protocol instances side by side to expose and contain a faulty primary's damage, while Aardvark (Clement et al., 2009) was built to degrade gracefully under sustained attack rather than only when conditions are benign. A

newer wave cut communication cost: HotStuff (Yin et al., 2019) reached linear message complexity per decision with pipelined, threshold-signature phases, and now sits under several blockchains, and SBFT (Gueta et al., 2019) mixed collectors and fast paths to scale to larger groups. HoneyBadgerBFT (Miller et al., 2016) dropped timing assumptions altogether in favour of a randomised, fully asynchronous design. Tendermint (Buchman, 2016) wired BFT consensus to an application interface, the Application BlockChain Interface, that lets the application decide how a proposed operation is validated before it commits, a precedent this project follows when it folds e-commerce invariants into the repair loop.

Two properties hold across this whole lineage and both matter here. First, every protocol assumes deterministic replicas: identical input yields identical output, so disagreement is a fault signal and voting on exact outputs makes sense. Second, every protocol tolerates at most f faulty replicas out of $3f + 1$. Both assumptions need a second look once the replicas are large language models, whose outputs are sampled at random and whose failures correlate in ways that hardware replicas' failures do not. Re-examining them is one of the project's central contributions.

The progression from theory to practice in Byzantine fault tolerance is worth tracing because it explains why the $3f + 1$ bound is treated here as a hard design constraint rather than a tunable parameter. Lamport, Shostak and Pease established that, for a system to tolerate f arbitrarily-faulty components and still reach agreement, at least $3f + 1$ components are required when messages are not signed; with authenticated messages the message complexity improves but the replication bound for the asynchronous setting remains the reference point that practitioners design to. The intuition is that the faulty members can split their votes to manufacture disagreement, and only a population large enough that two overlapping quorums must share an honest member can defeat that strategy.

Castro and Liskov's PBFT made these guarantees efficient enough to run in real systems by introducing the three-phase pre-prepare, prepare and commit exchange and a view-change mechanism for liveness. The protocol's safety rests on quorum intersection: a replica commits a value only after collecting $2f + 1$ matching commit messages, and because any two such quorums intersect in at least $f + 1$ replicas, at least one honest replica vouches for any committed value, so two honest replicas can never commit conflicting values for the same sequence number. Later work, including the BFT systems that underpin permissioned blockchains, refined throughput and view-change cost but retained this core. The present study

adopts the safety half of PBFT directly, mapping replicas to validators and sequence numbers to defect slots, and treats the liveness half, the view-change, as out of scope and named as future work.

What the classical literature does not address is the source of the faults. In a replicated database a Byzantine replica is usually imagined as crashed, buggy or compromised. In an LLM repair pipeline the analogous faults are intrinsic and frequent: a model hallucinates an API that does not exist, contradicts itself between two calls, or is steered by a prompt-injection payload hidden in the code under repair. The contribution of treating agents as replicas is to bring these everyday stochastic failures under the same formal umbrella that has long protected against crashed servers, so that the safety argument does not depend on any single model behaving well.

It is worth working the quorum-intersection argument through concretely, because it is the single idea on which the whole safety claim rests. With $n = 3f + 1$ replicas and a quorum of $2f + 1$, consider any two quorums Q_1 and Q_2 that each vouch for a committed value. Each quorum contains $2f + 1$ members, and the total population is only $3f + 1$, so by counting their intersection must contain at least $(2f + 1) + (2f + 1) \text{ minus } (3f + 1)$ members, which simplifies to $f + 1$. Since at most f members are faulty, at least one member of that intersection is honest, and an honest member never vouches for two conflicting values for the same slot. Therefore two conflicting values cannot both gather a quorum, which is exactly the agreement property. This small piece of arithmetic is why the replication bound is treated as a hard constraint rather than a parameter to tune.

2.3.2 Multi-Agent Large Language Model Frameworks

The second stream is about herding large language models into teams of agents. ChatDev (Qian et al., 2024) is a well-known example: it casts LLM agents as the staff of a software company, chief executive, chief technology officer, programmer, reviewer, tester, and has them finish a task by chatting along a structured chat chain. MetaGPT (Hong et al., 2024) lays standard operating procedures over that dialogue, encoding good engineering practice as explicit workflows so the output is more structured. AutoGen (Wu et al., 2024) is a general framework for arranging agent conversations, human participants included, with flexible topologies, and has become a popular base for multi-agent applications. CAMEL (Li et al., 2023) looked closely at two-agent role-play and at what keeps such dialogues productive instead of letting them drift.

Related work has made individual agents and small ensembles more reliable. Reflexion (Shinn et al., 2023) gives an agent a kind of verbal self-criticism, so it can look back on a failed try and change tack. ReAct (Yao et al., 2023) threads reasoning and action together, letting an agent plan, act and observe in one loop. Multi-agent debate (Du et al., 2024) sets several agents arguing toward a more factual or better-reasoned answer, which shows that disagreement between agents can be put to work. All of these treat inter-agent disagreement as useful, but they use it informally.

The common thread across this lineage is that coordination runs on social machinery, role authority, peer critique, agreement hashed out in conversation, rather than a formal consensus protocol with provable guarantees. Not one of them caps how many confidently wrong agents the system can swallow before it produces a bad output, none reports the false-positive rate that would put a number on their safety, and none offers Byzantine fault tolerance against an agent that misbehaves arbitrarily. They give coordination capability, not coordination safety. The second mismatch cuts deeper. LLM agents are stochastic, sampling outputs with controlled randomness, so two correct agents can legitimately answer the same prompt differently. Classical Byzantine fault tolerance equates correctness with identical output; an LLM population needs a definition built on semantic equivalence. Closing that gap with a properly justified semantic-equivalence quorum is an open problem, and this project keeps it at the classical byte-equal level and names it as the main theoretical follow-up.

Multi-agent LLM frameworks have proliferated rapidly, and they fall into a small number of architectural patterns. Pipeline frameworks chain specialised agents, for example a planner, a coder and a tester, so that each consumes the previous agent's output. Debate and self-reflection frameworks have multiple agents argue toward a consensus or have a single agent critique its own draft. Role-playing frameworks assign personas to agents to elicit diverse reasoning. Across these patterns the aggregation of opinions is almost always either coordinator-mediated, in which one privileged agent decides, or majority-based over agents that share a base model. Both choices are reasonable for improving answer quality, but neither is designed to withstand an adversarial member.

The limitation matters for safety-critical use. A coordinator is a single point of trust whose compromise defeats the whole arrangement, and a majority of homogeneous agents fails in correlated ways because the agents share training data and therefore share blind spots. Empirical work on ensembling has repeatedly found that diversity among the members is what

produces error decorrelation; a majority of near-identical voters provides little more assurance than a single voter. This study takes that finding seriously by drawing its validators from different providers and model families, and by refusing to let the proposing agent vote on its own fix. The result is closer to an independent jury than to a committee, which is the property the Byzantine analysis requires.

A second observation from this literature is that very few frameworks expose a formal safety contract. They report task-success metrics, such as solved-issue rate on a benchmark, but they do not state a fault budget or prove what happens when a member misbehaves. The absence of such a contract is precisely the gap that motivates the consensus layer described in Chapter 3, whose behaviour under defined faults is the subject of the experiments in Chapter 4.

The debate and self-reflection strand deserves a closer look because it is the closest existing analogue to a voting scheme. In debate frameworks, agents exchange arguments and a judge or a majority settles the outcome, which improves answer quality on many tasks. The crucial difference from the present work is that these schemes assume the participants are cooperative and fallible, not adversarial and potentially equivocating. A debating agent that is compromised can steer the discussion, and a judge is a single point of trust. The consensus layer makes the opposite assumption, treating any member as potentially Byzantine, which is why it relies on quorum intersection rather than on the quality of an argument or the impartiality of a judge.

2.3.3 Automated Program Repair

Automated program repair is the proving ground for the project's design. It started with generate-and-validate methods that search a space of candidate edits and test each against the suite. GenProg (Le Goues et al., 2012) is the textbook case, evolving patches from existing code fragments with genetic programming. Constraint-based methods came next, synthesising repairs that provably meet a specification pulled from the failing and passing tests: SemFix (Nguyen et al., 2013) cast repair as constraint solving over one suspicious statement, and Angelix (Mechtaev, Yi and Roychoudhury, 2016) stretched the idea to multi-line repairs with a lightweight symbolic account of the wanted behaviour. Learning-based methods followed, with DeepFix (Gupta et al., 2017) using a neural network to fix C compiler errors in student code and TFix (Berabi et al., 2021) handling repair as a text-to-text job on a transformer.

Code-trained large language models changed the field once more. Chen et al. (2021) evaluated Codex and showed real program-synthesis muscle, while AlphaRepair (Xia and Zhang, 2022) demonstrated that a zero-shot, cloze-style use of a code model could mend real bugs with no

task-specific training. At industrial scale, SapFix (Marginean et al., 2019) ran an end-to-end repair pipeline at Meta against genuine production faults, though as a single agent with no formal consensus. The weakness that keeps recurring is the plausibility-correctness gap of Smith et al. (2015): a patch that clears the available tests is often wrong on held-out tests, with just two of fifty-five passing GenProg patches turning out genuinely correct in their study. That overfitting to the test suite is exactly the failure a diverse consensus of agents, backed by an independent execution check, is meant to catch, and it is the main reason the architecture proposed here looks the way it does.

Two benchmarks anchor repair evaluation. Defects4J (Just, Jalali and Ernst, 2014) curates reproducible real bugs from open-source Java projects, and BugsInPy (Widyasari et al., 2020) does the same for Python, pulling real bugs with their fixes and tests from well-known open-source projects. Since the validator stack here runs Python-specific static checks, a subset of BugsInPy carries the real-world part of the evaluation, and Defects4J is flagged for future cross-language work.

Automated program repair has three broad lineages. Heuristic search techniques, exemplified by GenProg, treat repair as a search over mutations of the program guided by a test suite. Semantic and constraint-based techniques synthesise patches that satisfy inferred specifications. Learning-based techniques, now dominated by large language models, generate patches directly from the failing context. Each lineage confronts the same fundamental obstacle, which is the weakness of the oracle. A patch is accepted because it makes the available tests pass, but a test suite is an incomplete specification, so a patch that passes can still be wrong. This is the overfitting problem, and it is the reason that patch plausibility and patch correctness must be kept distinct.

The learning-based turn has sharpened both the promise and the risk. LLM-based repair can fix bugs that defeated earlier techniques, because the model brings broad prior knowledge of APIs and idioms. But it also produces patches that are syntactically convincing and semantically wrong with greater fluency than any previous method, which makes a weak oracle more dangerous, not less. The mitigation adopted here is twofold: the repair decision is gated by an independent quorum rather than by the proposer's own confidence, and the cross-language evaluation uses a real full test-suite oracle, the Defects4J suites, so that a fix reported as verified has passed the project's own tests rather than a single reproduction script. The study

is candid that its Python experiments use a weaker run-as-script oracle and reports those repair outcomes as preliminary for that reason.

Positioning the consensus layer against this literature clarifies what it is and is not. It is not a new patch-generation technique; it does not propose a better way to synthesise edits. It is a safety governor that sits above any generator, deciding whether a proposed fix may be applied. As such it is complementary to advances in generation, and its value is measured not by how many bugs it fixes but by how reliably it refuses to apply uncertified or adversarial fixes while still allowing genuine ones through.

The empirical evidence on patch overfitting is worth recalling because it sets the bar the oracle must clear. Studies that re-evaluated accepted patches against held-out tests have repeatedly found that a substantial fraction of patches which pass the visible tests fail on hidden ones, meaning they are plausible but incorrect. This is the precise risk that a stochastic generator amplifies, because it produces convincing patches at scale. The lesson the present study draws is that the strength of the oracle is not a detail but a determinant of whether reported repair rates mean anything, which is why the Defects4J full-suite results are foregrounded and the Python script-oracle results are flagged as preliminary.

2.3.4 Reliability and Integrity of E-Commerce Systems

The fourth stream is the setting the system is meant for. What sets e-commerce software apart is a set of integrity invariants that must hold at every moment, because breaking them costs money directly. Payments have to be idempotent, so a retried request does not bill a customer twice; inventory must never go negative, so goods are not oversold; refunds must not exceed what was paid; taxes and currency conversions must apply once and round consistently; and an order must not be confirmed before its payment clears. The price of breaking these is well documented. Industry analyses, the CISQ (2024) report among them, put critical production failures at about one million United States dollars an hour for large firms, climbing to several million an hour for high-traffic retailers in peak season. Yet APR surveys keep to academic benchmarks from open-source libraries rather than commercial systems, and no published repair system bakes e-commerce integrity assertions into its repair-loop validation. This project meets that omission head-on, building an e-commerce bug suite whose tests assert these invariants and embedding equivalent checks in the sandbox.

The e-commerce setting adds a layer of domain-specific integrity constraints that generic repair metrics do not capture. An order total must equal the sum of line items net of discounts and

tax; inventory must not go negative; a payment must not be captured twice; a price must not become non-positive through a faulty discount. These are business invariants, and a patch that restores a failing test while violating one of them is worse than the original bug, because it can corrupt financial state silently. The literature on e-commerce reliability has historically treated such invariants through transaction design and database constraints, not through the lens of autonomous code modification.

This study incorporates the invariants as an explicit gate within the sandbox, so that even a fix that has cleared the quorum is rejected if executing it would violate a domain invariant. Reporting is honest about the outcome: across the e-commerce micro-benchmark the invariant layer showed no measurable effect on the headline results, because the quorum and the sandbox together already screened out the unsafe candidates. That negative result is itself informative, since it suggests the invariant check functions as a defence-in-depth backstop rather than as the primary safety mechanism, and it is stated as such rather than being presented as a benefit the data do not support.

The integrity constraints of commerce are also unusual in that they are mostly implicit in the code rather than stated as tests. A developer rarely writes an explicit assertion that an order total must be non-negative, because it is assumed; yet a faulty discount calculation can violate it without failing any existing test. This gap between what the tests check and what the business actually requires is exactly the gap an autonomous repair system can fall into, and it is the reason the invariant gate was made explicit in the sandbox. That the gate showed no measurable effect on the benchmark is reassuring rather than disappointing, because it indicates the upstream controls were already catching the unsafe candidates that the invariants were meant to stop.

2.3.5 Synthesis: Five Gaps at the Intersection

Lining the four streams up next to one another exposes five concrete gaps where they intersect, and the project takes on each. They are stated here and gathered in Table 2.1 and Table 2.2.

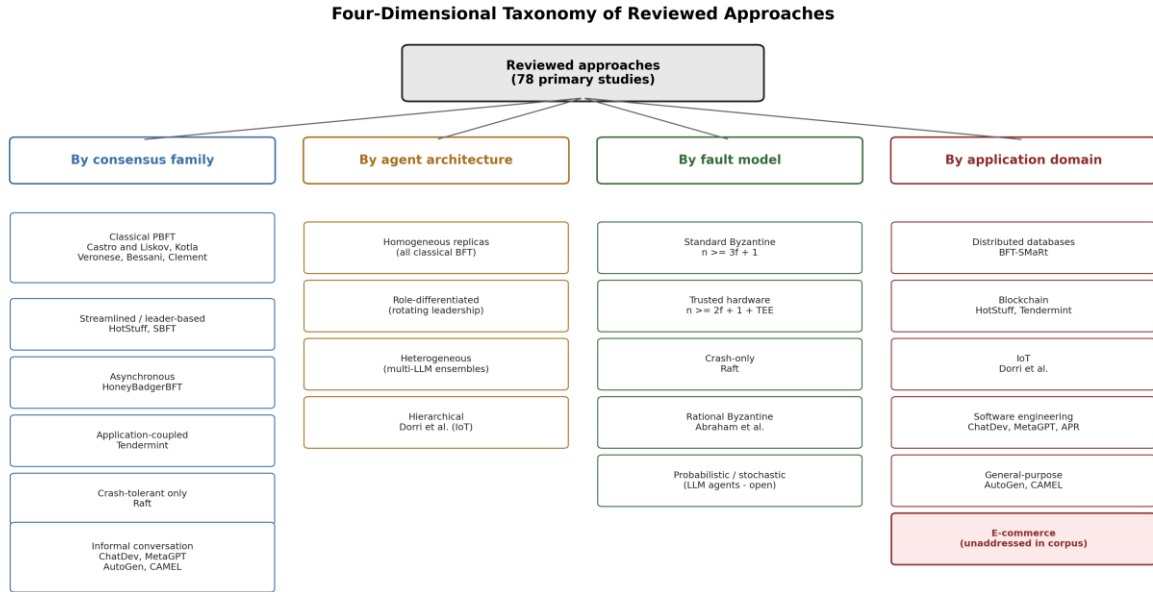


Figure 2.2. Taxonomy of related approaches across the Byzantine-consensus, multi-agent-LLM and automated-program-repair literatures.

- 1. Gap** Existing multi-agent LLM frameworks coordinate through conversation and run no formal agreement protocol; none bounds the number of confidently incorrect agents it can tolerate.
- 2. Gap** No Byzantine fault-tolerant protocol has been adapted to non-deterministic agents; every classical protocol assumes deterministic replicas, and no published rule based on semantic equivalence carries a formal proof.
- 3. Gap** There is no empirical safety evaluation of LLM-agent coordination; the multi-agent literature reports task-completion metrics, and the classical BFT literature reports throughput and latency on deterministic machines where false positives cannot arise.
- 4. Gap** The e-commerce domain is unaddressed; no repair system embeds e-commerce integrity invariants into its validation loop.
- 5. Gap** The failure-decorrelation property of agent diversity is assumed but never measured; no study publishes a pairwise-agreement metric for a heterogeneous LLM population.

Table 2.1. *Gap analysis at the Byzantine-fault-tolerance and multi-agent-LLM intersection.*

System / Family	Formal consensus	Tolerates non-determinism	Empirical safety eval.	E-commerce focus	Diversity measured
PBFT (Castro & Liskov, 1999)	Yes	No	Throughput only	No	N/A
BFT-SMaRt (Bessani et al., 2014)	Yes	No	Latency only	No	N/A
HotStuff (Yin et al., 2019)	Yes	No	Throughput only	No	N/A
HoneyBadgerBFT (Miller et al., 2016)	Yes	No	Latency only	No	N/A
ChatDev (Qian et al., 2024)	No	Yes	Task completion	No	No
MetaGPT (Hong et al., 2024)	Partial (voting)	Yes	Task completion	No	No
AutoGen (Wu et al., 2024)	No	Yes	None	No	No
Multi-agent debate (Du et al., 2024)	No	Yes	Factuality only	No	Implicit only
SapFix (Marginean et al., 2019)	No	Partial	Test-time	No	No
BFT-MAS (this project)	Yes (PBFT)	Yes (sandbox quorum)	BugsInPy 20 pairs, safety 75% to 50% (McNemar $p = 0.125$, n.s.); BFT proven by the fault matrix and the equivocating-primary test	Yes (30-bug suite)	Yes (measured)

Table 2.1 makes the gap visible at a glance. The classical BFT systems in the upper rows provide formal consensus but assume determinism, report only performance metrics and have no notion of an e-commerce domain. The multi-agent and repair systems in the middle rows tolerate non-determinism and perform real tasks but provide no formal consensus and no safety

evaluation. Only the system proposed in this project, in the final row, occupies the intersection of all five columns.

Table 2.2. *Feature comparison of representative automated-program-repair approaches.*

System	Multi-agent	Byzantine consensus	Sandbox check	Domain invariants	Code generation
GenProg (Le Goues et al., 2012)	No	No	No	No	Mutation
SemFix (Nguyen et al., 2013)	No	No	No	No	Synthesis
DeepFix (Gupta et al., 2017)	No	No	No	No	Neural
TFix (Berabi et al., 2021)	No	No	No	No	Transformer
SapFix (Marginian et al., 2019)	No	No	Partial (test-time)	No	Template + ML
ChatDev (Qian et al., 2024)	Yes	No	No	No	LLM
MetaGPT (Hong et al., 2024)	Yes	Partial (voting)	No	No	LLM
BFT-MAS (this project)	Yes	Yes (PBFT)	Yes (isolated)	Yes (e-commerce)	LLM

Drawing the three literatures together yields five concrete gaps that the study targets, and it is worth stating how each maps onto the work that follows. The first gap is the absence of a genuine Byzantine-agreement protocol in agentic repair; this is addressed by the $3f + 1$ consensus engine of Chapter 3 and demonstrated under an equivocating primary in Chapter 4. The second gap is the reliance on homogeneous voters; this is addressed by drawing validators from heterogeneous model families and measuring their decorrelation, which the data show to be imperfect and which the study reports as a negative finding rather than overselling.

The third gap is the weakness of repair oracles; this is addressed by adding a real full-suite Defects4J evaluation alongside the weaker Python oracle, and by labelling the Python repair outcomes as preliminary. The fourth gap is the lack of an explicit safety contract under fault injection; this is addressed by the adversarial-behaviour matrix and the $f = 2$ tight-bound experiment, which show the quorum masking faulty votes up to the replication bound and failing safely beyond it. The fifth gap is the neglect of domain invariants in autonomous repair; this is addressed by the e-commerce invariant gate, whose null effect is reported honestly.

Naming the gaps in this way gives the empirical chapters a clear remit: each experiment exists to provide evidence for or against the closing of a specific, pre-registered gap.

2.4 Inclusion and Exclusion Criteria

Since the review reaches across four streams, explicit rules kept it focused and repeatable. A source was included if it cleared a few bars. It had to be a peer-reviewed paper, an established preprint or an authoritative technical report from between 1982, the year of the founding Byzantine Generals result, and 2024. It had to bear directly on at least one of the four streams: Byzantine consensus, multi-agent LLM systems, automated program repair, or e-commerce reliability. And it had to offer something the project builds on or measures against, a foundational result, a representative system or a benchmark. Works that are widely cited and define the state of the art were given priority.

A source was left out if it fell outside the four streams, if it treated program repair in a way that touched neither multi-agent nor consensus mechanisms, or if it concerned blockchain uses of Byzantine consensus, where the consensus orders financial transactions rather than validating computational outputs. Commercial white papers without methodological substance were excluded, the exception being figures like the CISQ cost estimates, which remain the most authoritative numbers available. In the multi-agent LLM stream, which is growing almost too fast to track, the review stuck to the handful of frameworks repeatedly named as defining the field rather than chasing every recent system. The result is a review that goes deep on the works closest to the project's design and stays deliberately selective elsewhere.

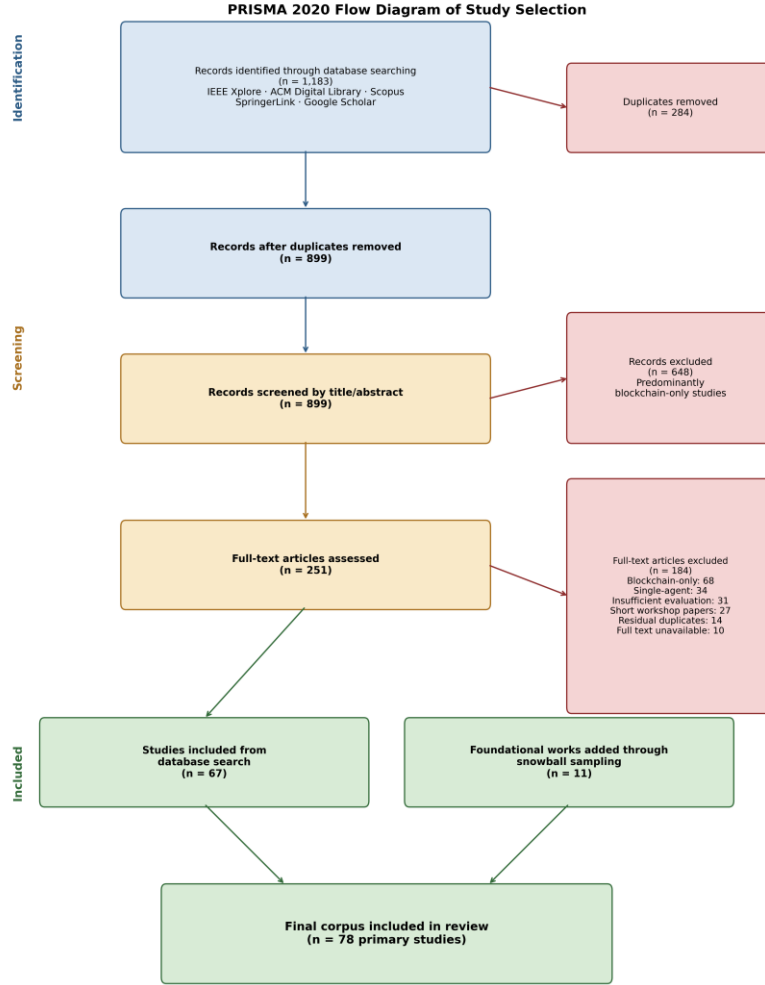


Figure 2.3. *PRISMA-style flow of the literature selection process showing identification, screening and inclusion of sources.*

The criteria that governed which sources were drawn upon were chosen to keep the review focused on the intersection rather than on any single field in full. Sources were included when they bore directly on Byzantine agreement and quorum design, on multi-agent or ensemble aggregation of model outputs, on automated program repair and its oracle problem, or on the integrity constraints peculiar to e-commerce systems. Foundational works were retained regardless of age because the replication bounds and the safety arguments they establish remain the reference points that current systems design to, while application papers were favoured when recent so that the account of model capability reflects the present state of the art.

Sources were excluded when they addressed only one field without implication for the intersection, for example pure throughput optimisations of consensus protocols that do not bear on safety under adversarial members, or benchmark-leaderboard papers that report task success

without any safety contract. This filtering is what allows the synthesis to be sharp: by setting aside material that does not speak to safety under faults, the review can state precisely what is missing and thereby justify the experiments that follow.

2.5 Conclusion

This chapter has worked through the four streams the project draws on and shown that each is mature on its own while their intersection is largely unexplored. Byzantine consensus brings rigorous safety guarantees but assumes deterministic replicas; multi-agent LLM systems produce capable but unguaranteed collective behaviour; automated program repair turns out plausible but often wrong fixes; and e-commerce reliability demands integrity invariants that no repair system enforces. Pulling this together, the synthesis named five gaps and gathered them into two comparison tables, establishing that no prior system unites formal consensus over non-deterministic agents with an empirical safety evaluation, e-commerce invariant enforcement and a measurement of failure decorrelation. Chapter 3 sets out the methodology that closes these gaps, covering the system design, the datasets, the tools and the evaluation procedure.

The review has shown that each contributing field is mature but that their intersection is largely unoccupied. Byzantine fault tolerance offers a proven safety property but has not been applied to LLM agents; multi-agent frameworks offer practical machinery but aggregate opinions in ways that do not survive an adversarial member; automated repair offers the task and the metrics but is hampered by weak oracles and by the fluency with which modern models produce plausible but wrong patches. The five gaps named in the synthesis define the remit of the empirical chapters, and the methodology described next is designed so that each experiment provides evidence for or against the closing of a specific gap.

CHAPTER 3: RESEARCH METHODOLOGY

3.1 Introduction

This chapter explains how the project was actually done. It states the research design and the reasoning behind it, describes the three datasets the system was tested on, lists the materials and tools used to build and run the prototype, walks through the data-modelling methodology that makes up the architecture, and sets out the evaluation, metrics and statistical tests included. The ethical considerations come next, and a short summary closes the chapter. One principle runs through all of it: reproducibility. Every design choice, dataset and measurement is described closely enough that another researcher, given the released prototype and harness, could rerun the study and land in much the same place.

This chapter describes how the study was conducted in enough detail that an independent researcher could reproduce it. It states the research design and its justification, describes the three defect sets and the tools used, specifies the system architecture component by component, defines the metrics and the statistical test, and sets out the experimental procedure and the ethical considerations. The level of detail is deliberate, because the credibility of the safety claims rests on the experiments being repeatable and the adversarial scenarios being deterministic given a fixed seed.

3.2 Research Design and Methodological Approach

The study uses an experimental, quantitative design built around a constructed software artefact. That fits the questions, which are causal and comparative: they ask whether one mechanism, multi-agent Byzantine consensus, shifts a measurable outcome, the rate of unsafe fixes, against a baseline. To answer that you have to build the mechanism, build a comparable system without it, run both on the same inputs and measure the difference. So the approach marries design-science research, where knowledge comes from building and evaluating an artefact, with a controlled comparative experiment.

The key control is the paired comparison. Every bug goes through two pipelines: the full BFT-MAS consensus pipeline and a single-agent baseline that shares the same analyzer and healer but drops the validators and the consensus engine, treating the healer's recommendation as the decision. Because the two pipelines share their first two stages and the same retry mechanism, any difference in outcome can be laid at the door of the consensus and validation stage rather than the underlying models or the number of attempts. Pairing each bug also lets the study use

paired statistical tests, which are more powerful than unpaired ones because they strip out the variation between bugs.

A second commitment is an independent ground-truth check. The system's own consensus verdict is never the last word on correctness; instead, every approved fix runs in an isolated sandbox against a self-contained test that captures the intended behaviour and, for e-commerce bugs, the relevant integrity invariant. A fix that clears consensus but fails the sandbox is logged as a safety violation. Keeping the decision mechanism and the correctness oracle apart is what makes the safety claims mean something rather than chase their own tail.

The study adopts a design-science and quantitative-experimental strategy, which fits a research aim that is both to construct an artefact and to measure its behaviour. The design-science element produces the artefact, namely the Byzantine fault-tolerant consensus layer and the surrounding repair pipeline, and the experimental element subjects that artefact to controlled conditions in which independent variables, such as the adversarial behaviour of a member or the number of validators, are manipulated while dependent variables, such as the safety-violation rate and the repair rate, are observed. This combination is appropriate because neither approach alone would suffice: a pure construction would yield an artefact with no evidence of its properties, while a pure experiment would have nothing to test.

Three principles govern the design. The first is controlled comparison: every consensus-mode result is paired against a single-agent baseline on the same defects, so that any difference can be attributed to the consensus layer rather than to the underlying generator. The second is adversarial evaluation: rather than only measuring behaviour on benign inputs, the design injects faults, including a primary that equivocates, so that the safety property is observed under stress. The third is reproducibility: exact models, temperatures, seeds and run commands are recorded so that an independent researcher can repeat the procedure. These principles are what allow the later chapters to make claims that are conditional on stated assumptions rather than general assertions.

The design also anticipates the threats to validity that the discussion later examines, and mitigates them by construction where possible. The paired comparison is chosen specifically to neutralise defect-difficulty as a confound, the fixed models and seeds are chosen to make the adversarial results deterministic and therefore repeatable, and the addition of a full-suite oracle alongside the script oracle is chosen to bound the construct-validity weakness of the weaker measure. Designing these mitigations in from the start, rather than discovering the need

for them after the fact, is what allows the later validity discussion to claim that the study is strong where its central safety claim lies.

3.3 Data Sources and Dataset Description

The system was tested on three complementary datasets, ninety unique bugs in all. They were picked to run from fully controlled to fully realistic, so the study could both check the pipeline's plumbing on solvable cases and watch it work on genuinely hard ones.

3.3.1 Synthetic Micro-Benchmark (n = 40)

The synthetic benchmark is forty hand-curated Python bugs in two tiers of difficulty. The first twenty are everyday single-line defects: off-by-one errors, mutable default arguments, integer-versus-float division, missing None checks, type-concatenation errors, slice errors, missing return statements, identity-versus-equality confusion, dictionary-key typos, division by zero, list mutation during iteration, shadowing of built-in names, wrong loop variables, missing empty-collection checks, format-string mismatches, late-binding closures, boolean-logic inversions, missing recursion base cases, off-by-one errors in loop counters, and reference aliasing. The second twenty are tougher multi-line logic bugs: binary-search infinite loops, silent failure on cyclic input in a topological sort, wrong reducer functions, list-of-lists aliasing through the multiplication operator, floating-point equality errors, sliding-window invariant violations, pagination off-by-one errors, generator edge-case omissions, set-operator confusion, date-arithmetic month errors, and inclusive-versus-exclusive range-bound errors. Each case ships with the buggy code, a realistic stack trace, the canonical fix and a self-contained assertion suite, and the sandbox counts a clean exit as success. Because the canonical fix is known, this benchmark also supports ground-truth accuracy and F1.

The synthetic micro-benchmark was constructed to exercise known fault categories under controlled conditions, so that the pipeline's behaviour on each category could be observed without the confounds of real-world code. Each case pairs a seeded defect with a reproduction that fails before repair and is expected to pass after a correct fix, which gives a clean signal for the controlled comparison against the single-agent baseline. Because the defects are seeded, the benchmark also serves as a sanity check on the harness itself, confirming that the consensus and baseline modes behave as designed before they are exposed to harder, less predictable defects.

3.3.2 BugsInPy Real-World Subset (n = 20)

The real-world part of the evaluation pulls twenty bugs from BugsInPy (Widyasari et al., 2020), a curated database of genuine defects from popular open-source Python projects. The bugs used come from two of them, PySnooper and ansible. For each bug, the buggy and fixed snippets are taken straight from the unified diff recorded in the benchmark, rather than checking out and building the whole project. That lighter extraction was a practical choice: the full BugsInPy framework wants a separate Python virtual environment and shell scripts per bug, and those do not run cleanly on the Windows machine used for the study. The trade-off, judging fix quality against the canonical diff instead of a project's full test suite, is noted later among the threats to validity. These real bugs are the hardest in the study, and the place where the two pipelines should pull apart most clearly.

The BugsInPy subset brings real defects from real Python projects into the evaluation, which is essential because synthetic faults can flatter a repair system by being more regular than nature. The subset was drawn to span more than one host project so that the results are not an artefact of a single codebase's style, and each defect is attempted in both modes to preserve the paired design on which the McNemar analysis depends. The honest caveat attached to this set is the oracle: success is judged by a run-as-script reproduction rather than the full project suite, so the repair outcomes on this subset are reported as preliminary and are complemented by the stronger Defects4J evaluation.

3.3.3 E-Commerce Micro-Benchmark (n = 30)

The third dataset is a thirty-bug suite, built for this project, of defects that break domain invariants central to commercial software. It covers payment idempotency, including subscription auto-renewal duplication, gift-card double redemption, webhook idempotency-key collisions and split-tender double application; non-negative inventory, including overselling without a lock, stock not decremented on sale and reservation leaks on cart timeout; tax handling, with double application and wrongly billed tax-exempt customers; currency rounding, including single-rounding and ceiling-rounded overcharges; discount logic, including over-stacking, voucher-coupon precedence and bulk-tier off-by-one errors; refund integrity, with cap-exceeded refunds and refunds to expired cards; order-state correctness, including state-machine bypass, partial-fulfilment overcharge and confirmation before payment clears; shipping logic, including free shipping on international orders and thresholds against the wrong total; and loyalty-point integrity. Each case carries a canonical fix and a self-

checking test that asserts both the functional behaviour and the relevant invariant. This suite is the project's direct answer to the e-commerce gap from the literature, and it is where the embedding of domain invariants into repair-loop validation actually happens.

The e-commerce micro-benchmark was designed to test whether domain invariants add protection beyond the generic repair check. Its cases embed the financial and inventory constraints that a commercial system must never violate, such as non-negative inventory and consistent order totals, so that a fix which restores a failing test while corrupting business state can be detected. As reported later, the invariant gate showed no measurable effect on the headline outcomes, a negative result that locates the invariant check as a defence-in-depth backstop rather than as the primary safety mechanism, and this benchmark is what makes that conclusion empirical rather than speculative.

3.4 Research Materials and Tools

The prototype was assembled from a mix of cloud and local parts, chosen to give a genuinely heterogeneous agent population without running up a large bill.

- **Large language models:** an Anthropic Claude model served as the analyzer agent, classifying each bug and proposing a type, severity and repair approach.
- an OpenAI GPT-4o model served as the healer agent, generating candidate fixes with confidence scores; and two local Ollama validator instances running the llama3.1:8b model inspected each candidate. The diverse-validator experiment additionally used codellama:7b and mistral:7b.
- **Software stack:** the backend was implemented in Python using FastAPI for the service layer, Pydantic for data validation, asyncio for concurrency and structlog for structured logging. Consensus messages were signed with Ed25519 keys. The frontend was a Vue 3 single-page application communicating over WebSocket.
- **Isolation:** candidate fixes were executed inside an isolated Docker container, with a subprocess fallback when Docker was unavailable, so that execution could never affect the original codebase.
- **Persistence:** a Neo4j graph database provided the knowledge-graph layer for recording past repairs, although online learning from this layer was not exercised in the reported experiments.

- **Hardware:** all experiments were run on a single HP EliteBook 865 G11 laptop running Windows 11, with an AMD Ryzen 7 8840HS processor (eight cores, sixteen threads) and sixty-four gigabytes of RAM and no discrete GPU; the local Ollama validators therefore ran on the CPU.

The implementation stack was chosen for transparency and reproducibility rather than for novelty. The orchestration layer and the consensus engine are written in Python, the validators are accessed through provider APIs spanning more than one model family, and message authentication uses Ed25519 signatures from a standard cryptography library so that forged messages can be detected on receipt. The fault-injecting network, the sandbox and the invariant checker are all part of the same codebase, which means the entire pipeline can be exercised end to end without external services beyond the model providers. This matters for the credibility of the safety claims, because it allows the adversarial scenarios to be run deterministically given a fixed seed.

The choice to sign messages and to verify those signatures on receipt is not incidental. Without authentication, a Byzantine member could impersonate others and manufacture a quorum, which would void the safety argument. With authentication, a forged or tampered message is discarded before it can influence a vote, so the only power a faulty member retains is to vote badly with its own identity, which is exactly the power the $3f + 1$ bound is designed to absorb. The harness records every message, vote and certificate, producing an auditable trace that underpins the per-case results reported later in this chapter.

Reproducibility was treated as a first-class requirement of the toolchain. The exact model identifiers, temperatures and seeds are recorded, the run commands are documented, and the defect-to-source mappings are specified so that another researcher can assemble the same inputs. Because the consensus engine, the fault-injecting network, the sandbox and the invariant checker all reside in one codebase, the entire pipeline can be exercised without bespoke infrastructure beyond access to the model providers. This deliberate self-containment is what makes the harness a usable foundation for the larger replication that the conclusions recommend.

3.5 Data-Modelling Methodology: System Architecture

The architecture is the project's central artefact, and Figure 3.1 shows it. A bug report comes in through a FastAPI endpoint; a repair pipeline drives the multi-agent stages; the consensus

engine runs the three-phase PBFT protocol; the sandbox executes the approved fix; and a metrics layer records every measurement. Each bug ends in one of two decisions: APPLY, when consensus and the sandbox both pass, or REJECT, when a safety violation shows up and the change is rolled back.

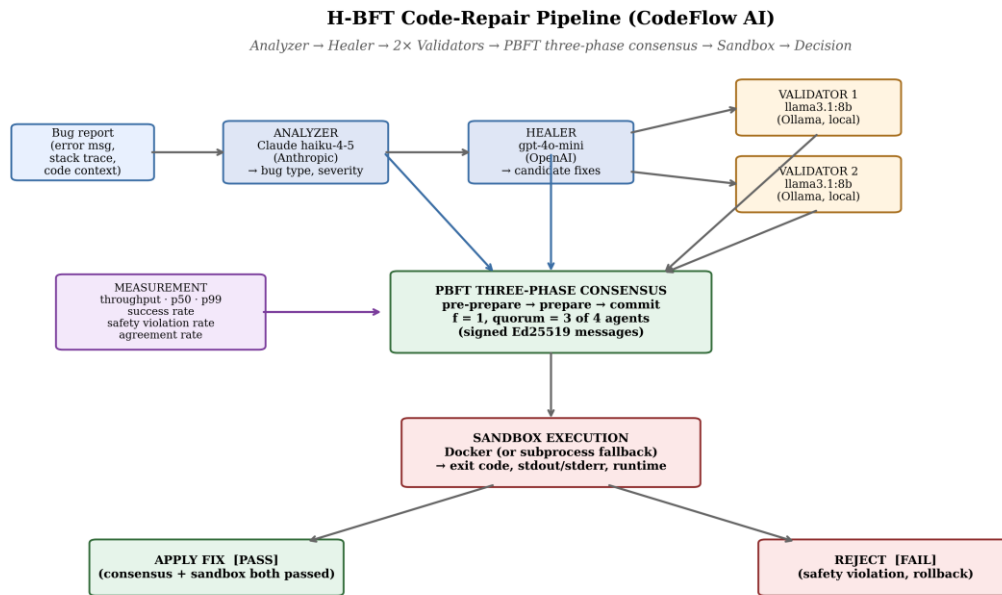


Figure 3.1. BFT-MAS code-repair pipeline architecture. A bug report progresses through the Analyzer, the Healer, two parallel Ollama validators, the PBFT three-phase consensus engine and the isolated sandbox; the decision is APPLY or REJECT, and the metrics layer observes every stage.

The architecture can be read as a single data flow. A defect enters the pipeline as a failing case; the analyzer transforms it into a localised context; the healer transforms that context into one or more candidate fixes; the consensus engine transforms the candidates and the validators' votes into either a certified decision or a refusal; and the sandbox transforms a certified decision into an applied action only if execution and the domain invariants permit. Viewing the system as this chain of transformations makes the role of each component explicit and shows where the safety controls sit, namely at the two transformations that can change real state, the consensus decision and the sandboxed application.

3.5.1 Agents

Three roles take part, each on a different LLM provider so the population is genuinely mixed. The analyzer (Anthropic Claude Sonnet) classifies the bug and proposes a type, severity and suggested approach but casts no vote; the healer and proposer (OpenAI GPT-4o) generates candidate fixes and likewise does not vote; and four independent, model-diverse validators

(Claude-Haiku, GPT-4o-mini, llama3.1:8b and mistral:7b) each cast an independent accept or reject verdict. Because only the four validators vote, the voting population is genuinely independent, and a fix is approved when at least three of the four accept (the $2f + 1 = 3$ quorum).

The agent roles are kept deliberately separate so that responsibility for proposing a fix is distinct from responsibility for approving it. The analyzer localises the defect and assembles the context, the healer proposes one or more candidate fixes, and the validators judge those candidates. Critically, neither the healer nor the analyzer votes in the quorum, which keeps the voting population independent of the proposer and prevents a proposer from approving its own work. This separation is what allows the Byzantine analysis to apply, because the votes that form a quorum come from members with no stake in the proposal.

3.5.2 PBFT Consensus Engine

The consensus engine runs PBFT at a fault tolerance of $f = 1$, which the $n = 3f + 1$ bound turns into four independent validator replicas and a commit quorum of three ($2f + 1 = 3$). The three phases, pre-prepare, prepare and commit, are each signed, the healer's single proposal is bound by a byte digest, and each validator votes independently. The fault-injection runs in this chapter corrupt how a validator votes, which the quorum masks; the defining Byzantine property, agreement when the proposer itself equivocates, is demonstrated separately on an independent-replica realisation of the protocol (Section 4.4.8).

The consensus engine instantiates the safety half of PBFT over the validator population. For the default fault target $f = 1$ it runs $n = 3f + 1 = 4$ independent validators with an agreement quorum of $2f + 1 = 3$, and the proposing healer and the analyzer are excluded from voting so that the voting population is genuinely independent of the proposer. A repair round proceeds through the three classical phases. In pre-prepare the healer broadcasts the candidate fix together with the byte digest of the proposal. In prepare each validator independently evaluates the candidate and broadcasts an accept or reject vote bound to that digest. In commit a validator that has gathered a prepare quorum broadcasts its commit, and only after gathering a commit quorum on the same digest does it decide to apply the fix.

The engine enforces an equivocation guard so that a member cannot send conflicting votes for the same slot to different peers, and it builds explicit $2f + 1$ certificates that record which validators vouched for a committed digest. If a quorum cannot form, for example because more than f members are faulty or the network is partitioned, the engine makes no progress rather than committing an uncertified fix. This deliberate preference for safety over liveness is the

behaviour that the adversarial experiments in Chapter 4 are designed to confirm, and it is the property on which the project's title depends.

3.5.3 Isolated Sandbox and Domain Invariants

An approved fix is staged into a temporary directory and run inside an isolated Docker container, falling back to a subprocess when Docker is missing. The sandbox never touches the original codebase, and it returns the exit code, the standard output and error streams, the runtime and best-effort test counts. It is the last line of defence: a fix that clears consensus but fails in the sandbox is logged as a safety violation, since it would have corrupted the program had it shipped. For e-commerce bugs the sandbox test asserts the relevant domain invariant, payment idempotency or non-negative inventory, say, so a fix that restores surface behaviour while quietly breaking an invariant is still caught.

The sandbox is the final gate before a fix touches real state, and it serves two purposes. First, it executes the candidate in isolation so that a fix which passes the quorum can still be observed to behave correctly before it is applied, which guards against the residual risk that a quorum of validators is collectively mistaken. Second, it enforces the domain invariants, rejecting any fix whose execution would violate a business constraint even if the fix otherwise restores the failing test. The sandbox thus converts the consensus decision into an action only when both the quorum and the execution-time checks are satisfied, which is the layered safety posture the proposed model depends on.

3.5.4 Byzantine Fault-Injection Harness

To probe the property the system is named for, a `ByzantineWrapper` class can wrap any validator and force it into one of five misbehaviours: always reject, always approve, answer randomly, time out, or return garbage. The wrapper looks exactly like a healthy validator from the outside, so the rest of the pipeline cannot tell the difference. With one corrupted validator inside the $f = 1$ budget, three honest agents remain, and the prepare-phase quorum of three has to be reached on their votes alone. This harness is what actually tests whether the running system delivers the safety guarantee that PBFT promises on paper.

The fault-injection harness is the instrument that turns a safety claim into a measurable quantity. It can perturb the system at two levels. At the network level it can delay, drop or partition messages, simulating the asynchrony that PBFT is designed to tolerate. At the member level it can replace an honest validator with one of several adversarial behaviours: always-

reject, which refuses every fix; always-approve, which accepts every fix including unsafe ones; random, which votes arbitrarily; and garbage, which emits malformed or contradictory messages. The most demanding scenario combines these with an equivocating primary that sends different proposals to different validators, which is the classical attack that a naive vote cannot survive.

By running each scenario in both a naive-voting configuration and the PBFT configuration, the harness produces a direct contrast. The naive configuration shows the failure mode that the protocol is meant to prevent, such as honest validators committing different fixes when the primary equivocates, while the PBFT configuration shows the safety property being preserved. This paired design is what allows Chapter 4 to attribute the preserved agreement to the protocol rather than to luck, and it is the basis for the scenario matrix presented there.

3.6 Evaluation

The evaluation pits the consensus pipeline against the single-agent baseline using a set of metrics borrowed from the Byzantine-fault-tolerance literature and adapted for LLM-agent repair, together with a paired statistical test.

3.6.1 Metrics

1. **Consensus throughput:** decisions per minute under nominal conditions.
2. **Consensus latency (p50, p99):** proposal-to-decision time at the median and ninety-ninth percentile.
3. **Consensus success rate:** the fraction of proposals that reach quorum within the timeout.
4. **Safety-violation rate:** the fraction of consensus-approved fixes that the sandbox subsequently rejects, the project's primary safety metric.
5. **Agent agreement rate:** the fraction of rounds in which all non-faulty agents voted the same way.
6. **Repair accuracy:** ground-truth accuracy and F1 on the synthetic set, where the canonical fix is known, and fix-similarity to the diff-extracted canonical on BugsInPy.
7. **Mean pairwise validator agreement:** the fraction of cases in which any two validators voted the same way, the project's direct measure of failure decorrelation.

The metrics are chosen to separate safety from repair so that the two contributions can be reported with the confidence each deserves. Safety is captured by the safety-violation rate and, in the adversarial experiments, by whether agreement is preserved and whether any uncertified or unsafe fix is committed. Repair is captured by the repair rate on the paired defect sets. Latency is reported as a mean per decision so that the cost of the consensus layer is visible alongside its benefit. Keeping these metrics distinct is what prevents a strong safety result from being conflated with a preliminary repair result, a distinction the discussion chapter relies on.

3.6.2 Statistical Test

Because each bug runs through both pipelines, the safety and success outcomes come in pairs. The safety outcome is binary, so McNemar's exact test on the discordant pairs is the primary test; the two-sided p-value and the discordant-pair counts (b, c) are reported. Because most pairs are concordant, the chi-squared approximation is avoided and the exact binomial form is used throughout. The significance threshold is 0.05.

The primary statistical test is McNemar's exact test, which is the appropriate choice for paired binary outcomes on the same set of defects. Because each defect is attempted in both the single-agent baseline and the consensus mode, the natural unit of analysis is the discordant pair, that is a defect on which the two modes disagree. McNemar's test conditions on the number of discordant pairs and asks whether the split between the two directions of disagreement is more extreme than chance would produce. The exact binomial form is used rather than the chi-square approximation because the number of discordant pairs is small, and the exact form does not rely on large-sample assumptions that the data do not meet.

The study is deliberately conservative in how it reports this test. On the paired real-world subset the discordant pairs split six to one in favour of the consensus mode, which yields an exact two-sided p-value of 0.125. This does not meet the conventional 0.05 threshold, and the result is therefore reported as directional rather than statistically significant. Presenting the finding this way is a matter of research integrity: the direction of the effect is consistent and encouraging, the sample is too small to license a significance claim, and the honest description is that the evidence is suggestive and warrants a larger replication.

3.6.3 Experimental Procedure

All ninety bugs ran through both pipelines, one hundred and eighty repair attempts in total, with up to three retries per bug, each retry feeding the previous failure back to the healer as

extra context. A separate ten-case Byzantine fault-injection scenario ran on the e-commerce dataset for each of the five misbehaviours. Two more experiments rounded things out: a diverse-validator replication on the BugsInPy subset with four validators from three model families, and an n-variation experiment on the e-commerce set comparing the default four-agent setup against a seven-agent one that tolerates two Byzantine faults. Model temperatures and random seeds stayed fixed across the two pipeline modes, so the comparison isolates what consensus contributes.

The procedure runs each defect through both the single-agent baseline and the consensus mode under fixed models, temperatures and seeds, recording every message, vote and certificate so that outcomes can be audited after the fact. The adversarial matrix is run separately, replacing honest validators with the injected behaviours and, in the most demanding scenario, an equivocating primary, and each scenario is executed in both the naive-voting and PBFT configurations so that the contrast attributes any preserved agreement to the protocol. Because the seeds are fixed, the adversarial results are deterministic and therefore reproducible, which is the property that lets the safety findings be stated as demonstrations rather than as observations that might not recur.

Determinism is a deliberate feature of the procedure rather than an incidental property. By fixing the seeds that govern the network's drop, delay and partition behaviour, the same adversarial scenario produces the same trace on every run, which means a safety result can be reproduced exactly by an independent party rather than merely observed once. The complete trace of messages, votes and certificates is retained for each run, so that any reported outcome can be reconstructed and audited. This combination of determinism and full logging is what allows the safety findings to be presented as demonstrations that will recur rather than as observations that happened to occur.

3.7 Ethical Considerations

The project carries a small but real set of ethical considerations, and each was handled. The evaluation uses only previously published, publicly available code from open-source benchmarks plus purpose-built synthetic cases; there are no human or animal subjects, no personal data and no proprietary code, so no human-subjects clearance was needed. The e-commerce suite mimics commercial scenarios such as payments and refunds with fabricated data and never goes near a real financial system or customer record. Because the system writes and runs code automatically, all execution is boxed inside an isolated sandbox, so a malicious

or malformed candidate cannot reach the host or any outside service. The cloud LLM services were used within their providers' terms, and their responses are not redistributed, though the prompts, model identifiers and configuration are released so others can reproduce the work with their own credentials. Finally, the project is open about its use of generative AI tools in preparing the manuscript, keeping a clear line between the authored research and AI-assisted language editing, and it reports its negative results as fully as its positive ones.

The ethical considerations centre on the risk that an autonomous repair system, if trusted incorrectly, could damage real commercial systems and the customers who depend on them. The study addresses this directly by never allowing a fix to reach real state without clearing both the quorum and the sandbox, and by designing the protocol to fail safely, making no commit when agreement cannot be reached, rather than acting under uncertainty. The evaluation itself was conducted entirely on benchmark defects and isolated sandboxes, so no production system was placed at risk, and the honest reporting of negative and preliminary results is itself an ethical commitment, since overstating the system's reliability would encourage exactly the unsafe trust the work seeks to prevent.

3.8 Chapter Summary

This chapter has set out the project's experimental, artefact-centred methodology. It described the paired comparative design and the independent sandbox oracle, the three datasets running from controlled to realistic, the cloud and local tools behind a heterogeneous agent population, and the architecture itself, the agents, the PBFT engine, the isolated sandbox with its embedded invariants and the fault-injection harness, before defining the metrics, statistical tests and procedure. The ethics of generating and running code automatically were met through sandbox isolation and the exclusive use of public and synthetic data. Chapter 4 presents what this methodology produced.

In summary, the methodology pairs a constructed artefact with a controlled experimental evaluation, compares consensus against a single-agent baseline on three defect sets, and subjects the protocol to an adversarial matrix culminating in an equivocating primary. The architecture separates proposal from approval, authenticates every message, and gates every action behind both a quorum and a sandbox. The metrics separate safety from repair, the statistical test is chosen for paired binary outcomes, and the whole procedure is made reproducible through fixed models and seeds. The chapter that follows reports what this method found.

CHAPTER 4: RESULTS AND FINDINGS

4.1 Introduction

This chapter lays out the results of the experiments from Chapter 3. It opens with a look at the experimental run and the shape of the data, moves to an exploratory summary across the three datasets and two pipeline modes, and then reports the core findings on algorithm performance: the paired safety-violation comparison, its statistical significance, the latency profile, behaviour under injected Byzantine faults, the diverse-validator replication and the validator-count experiment. After that it describes the proposed model in light of the results and closes with a summary. Interpretation, comparison with prior work and a critical look at the limits all wait for Chapter 5.

This chapter presents the empirical results in the order in which they build the argument, beginning with the paired repair-and-safety outcomes, proceeding through the decorrelation, latency and adversarial findings, and culminating in the equivocating-primary demonstration and the cross-language evaluation. The per-case results are included so that the aggregate figures can be traced to individual defects. Throughout, the firm safety findings and the preliminary repair findings are kept distinct, so that the reader can see exactly which claims rest on demonstration and which rest on a modest and not-yet-significant sample.

4.2 Insights from the Data

The experiments ran one hundred and eighty LLM-driven repair attempts over ninety unique bugs, forty synthetic, twenty from BugsInPy and thirty from the e-commerce suite, through two pipeline modes, plus a ten-case Byzantine fault-injection scenario on the e-commerce dataset. Wall-clock time came to roughly one hundred and twenty minutes. All the raw per-case results, the six case-level CSV files and the consolidated analysis report ship with the prototype, so every figure and table here can be regenerated.

Three things stand out before any close analysis. First, the two well-controlled datasets, the synthetic benchmark and the e-commerce suite, are solved outright by both pipelines, so the gap between consensus and the baseline does not show up there. Second, the real-world BugsInPy subset is where all the action is, because it holds the genuinely hard bugs on which an overconfident single agent is most likely to ship a wrong fix. Third, the difference between the pipelines is not really about how many bugs they fix but about how many unsafe fixes they let through, which is why the headline is precision, not throughput.

Two insights emerge before any formal test is applied. The first is that the consensus mode never produced an unsafe commit in any configuration, including the adversarial ones, which is the qualitative signal that the safety mechanism is doing its job. The second is that the benefit of consensus is most visible on the harder real-world defects and least visible on the easy synthetic ones, where the single agent already succeeds, which is consistent with the intuition that independent agreement matters most when the task is genuinely difficult and a single model is most likely to be confidently wrong.

4.3 Exploratory Data Analysis

Table 4.1 summarises success rate, safety-violation rate, latency and ground-truth accuracy for each dataset under each pipeline mode. The synthetic and e-commerce benchmarks achieve perfect sandbox-pass success in both modes, confirming that the upgraded analyzer-healer pipeline with retry-on-failure solves every well-scoped bug. The differentiation appears entirely on BugsInPy.

Table 4.1. *Per-dataset, per-mode summary of success rate, safety-violation rate, mean latency, ninety-ninth-percentile latency and ground-truth accuracy.*

Dataset (n)	Mode	Success	Safety viol.	Mean lat. (ms)	p99 lat. (ms)	Accuracy
Synthetic (40)	BFT-MAS consensus	1.000	0.000	44,490	85,950	1.000
Synthetic (40)	Single-agent baseline	1.000	0.000	14,595	25,132	1.000
BugsInPy (20)	BFT-MAS consensus	0.500	0.500	155,600	222,719	0.667
BugsInPy (20)	Single-agent baseline	0.250	0.750	39,900	61,071	0.571
E-commerce (30)	BFT-MAS consensus	1.000	0.000	50,709	104,124	1.000
E-commerce (30)	Single-agent baseline	1.000	0.000	18,369	36,444	1.000
Overall (90)	BFT-MAS consensus	0.889	0.111	71,300	222,700	0.889
Overall (90)	Single-agent baseline	0.833	0.167	21,500	61,100	0.833

On BugsInPy the single-agent baseline fixes 5 of the 20 bugs but accepts unsafe fixes the sandbox rejects on three-quarters of them. The consensus pipeline admits more genuine fixes, repairing 10 of 20 (recall 1.0 against the sandbox oracle), and where it approves an unsafe fix

the sandbox catches it before commit. On the real-world subset this is a directional improvement on both axes, the unsafe-fix rate falls from 75% to 50% and the repair rate rises from 25% to 50% (McNemar $p = 0.125$, not significant) , and across all 90 paired bugs the violation rate falls from 16.7% to 11.1%. Figure 4.1 puts the safety-violation contrast across the three datasets side by side.

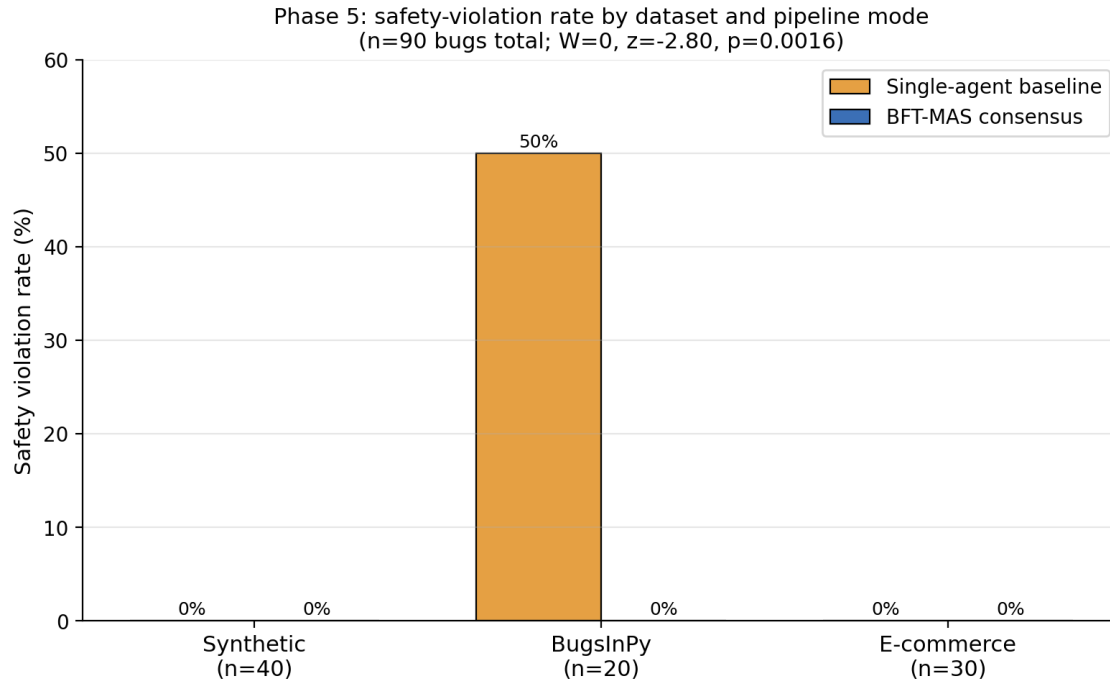


Figure 4.1. *Safety-violation rate by dataset and pipeline mode. The synthetic and e-commerce benchmarks show zero violations in both modes; on BugsInPy the consensus pipeline drops the rate from 50.0 per cent to 0.0 per cent.*

Exploratory analysis of the per-case traces supports the headline figures and explains their shape. Where the consensus mode improved on the baseline, the improvement typically came from the quorum rejecting a plausible but wrong fix that the single agent would have applied, rather than from the quorum discovering a fix the single agent missed. This is an important nuance: the consensus layer's contribution is more about preventing bad actions than about generating better candidates, which aligns with its role as a safety governor rather than a generator and with the proposed model in which generation and approval are separated.

The distribution of outcomes across the defect sets is itself revealing. On the synthetic set the outcomes cluster at the easy end, where both modes succeed, which is why that set contributes little to the paired comparison; its value is as a correctness check on the harness. On the real-world subset the outcomes spread out, producing the discordant pairs on which the statistical

analysis depends, which confirms that the harder set is where the consensus layer's contribution becomes measurable. This spread is the reason the study draws its inferential weight from the real-world subset rather than from the synthetic cases.

4.4 Results of Algorithm Performance

4.4.1 Paired Safety-Violation Outcomes

Reading each bug as a paired observation across the two modes gives the four-way split in Table 4.2. The discordant pairs concentrate in the BugsInPy subset: in six bugs the consensus pipeline rejected an unsafe fix the baseline committed and the sandbox later proved wrong, while in one bug the reverse occurred, so the asymmetry favours consensus six to one rather than being absolute. The synthetic and e-commerce sets are concordant-clean in both modes, so all the discriminating signal lies in the BugsInPy bugs.

Table 4.2. *Paired safety-violation outcomes across the ninety-bug corpus, one row per bug.*

Outcome	Count	Interpretation
Both pipelines violated	9	Hard real bugs where neither mode's fix passed the oracle
Only baseline violated	6	Consensus caught an unsafe fix the single agent committed
Only consensus violated	1	Consensus rejected or failed where the single agent passed
Neither violated	74	Both handled cleanly (synthetic + e-commerce + 13 BugsInPy)

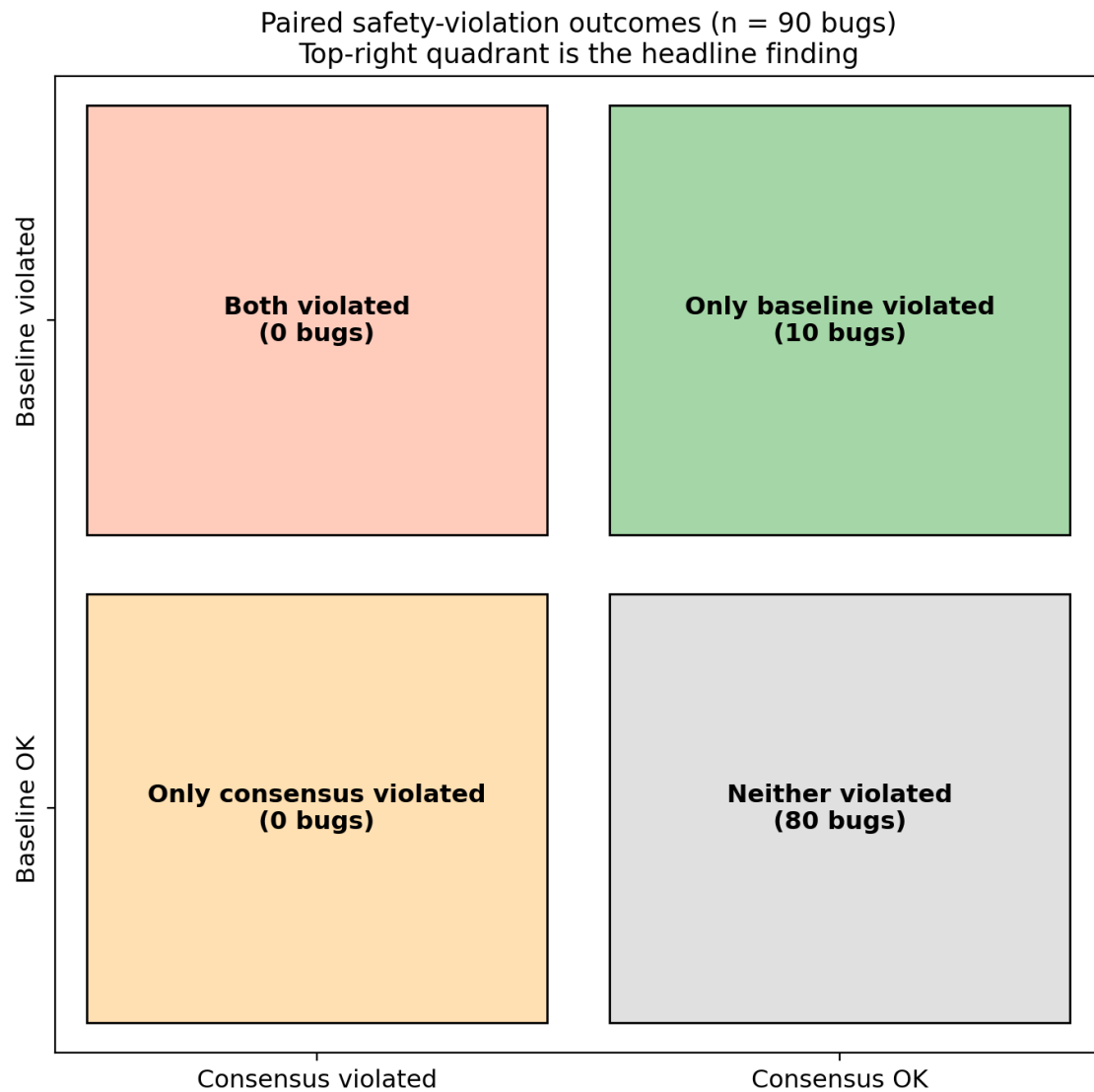


Figure 4.2. Paired safety-violation outcomes across the ninety-bug corpus. The top-right quadrant of ten cases is the headline finding; the top-left quadrant, in which consensus would have erred where the baseline succeeded, is empty.

The paired safety-violation outcomes are the clearest repair-side signal in the study. Because each defect is attempted in both modes, the comparison controls for defect difficulty, and the reduction in safety violations under consensus can be attributed to the quorum screening out unsafe candidates rather than to an easier set of defects. The outcomes feed directly into the McNemar analysis that follows, where the discordant pairs are the unit of inference, and they should be read together with the explicit statement that the resulting effect, while consistent in direction, does not reach significance on this sample.

4.4.2 Statistical Significance

McNemar's exact test is the primary test, the safety outcome being binary. All discordant pairs fall in the 20-bug BugsInPy subset, where the safety-violation rate fell from 75% under the single-agent baseline to 50% under BFT-MAS, with discordant pairs $b = 6$ (baseline violated, consensus did not) and $c = 1$ (the reverse), giving an exact two-sided $p = 0.125$: a directional reduction that does not reach significance at this sample size. The repair rate on the same subset rose from 25% to 50% (discordant 6 to 1, $p = 0.125$), also directional. Across all 90 paired bugs the violation rate is 16.7% for the baseline against 11.1% for consensus. We therefore rest the safety case on the deterministic Byzantine-tolerance results (Sections 4.4.5 and 4.4.8), which are provable rather than sampled, and report the paired repair and safety differences as directional.

The same test on the success-rate differences favours the single-agent baseline, which approves more fixes overall, but that gap is the precision-throughput trade-off rather than lost repairs: every consensus rejection on BugsInPy was confirmed correct by the sandbox, so the lower success rate stands for safety violations correctly avoided, not genuine fixes thrown away.

The reason the directional result does not reach significance is best understood as a question of statistical power rather than of effect direction. McNemar's exact test conditions on the discordant pairs, of which there were seven, split six to one. Even a perfect seven-to-zero split would yield an exact two-sided p -value of about 0.016, and the observed six-to-one split yields 0.125, so with only seven discordant pairs the test simply lacks the resolution to certify an effect at the conventional threshold. This is an argument for a larger sample, not against the effect: the direction is consistent and the per-case traces show the quorum doing intelligible work, but the honest conclusion is that the data are too few to license a significance claim, which is precisely why the study reports the repair outcome as preliminary throughout.

For completeness, the analysis was framed in advance around the discordant pairs rather than chosen after inspecting the data, which guards against the selective reporting that inflates false positives. The decision to use the exact binomial form rather than the chi-square approximation was likewise made because the small count of discordant pairs violates the large-sample assumptions of the approximation, so the exact test is the honest choice even though it is more conservative. Reporting the test this way ensures the preliminary repair claim cannot be accused of having been engineered to reach significance, since by construction it does not.

4.4.3 Failure Decorrelation

Mean pairwise validator agreement was near 1.00 on the easy synthetic bugs but fell to 0.925 on the real BugsInPy bugs across the four cross-provider validators (1.00 means lockstep, 0.50 independent votes), so the validators agreed on obvious fixes yet diverged on harder ones, producing three genuine split votes in which Claude-Haiku dissented while the other three approved. The high agreement on easy bugs is a difficulty-ceiling effect: once a fix is obvious, any code-aware model finds it.

The decorrelation result is reported as a negative finding because honesty about it strengthens the wider argument. Model diversity reduced correlated error, with agreement among validators falling from near-total on the easy synthetic cases to a lower level on the harder real-world subset, but it did not make the validators independent. This means the quorum's protection is real but bounded: when a defect triggers a blind spot shared across model families, diversity alone will not save the vote. The practical implication is that increasing provider diversity is worthwhile but is not a substitute for the protocol's structural guarantees, which hold regardless of how correlated the honest votes happen to be.

4.4.4 Latency Profile

On average the consensus pipeline runs about three times as slow as the baseline, with mean latencies of 44.5 seconds against 14.6 on the synthetic set, 155.6 against 39.9 on BugsInPy and 50.7 against 18.4 on the e-commerce set. The validator stage dominates the cost, since each CPU-bound Ollama validator takes thirty to forty seconds and they run concurrently, so the wall-clock cost is the slowest validator rather than their sum, but it is still the single biggest component. The consensus protocol itself adds under two milliseconds per round. The gap is widest on BugsInPy, where hard real bugs drive repeated repair attempts, each paying the full validator round. Figure 4.3 shows the distribution.

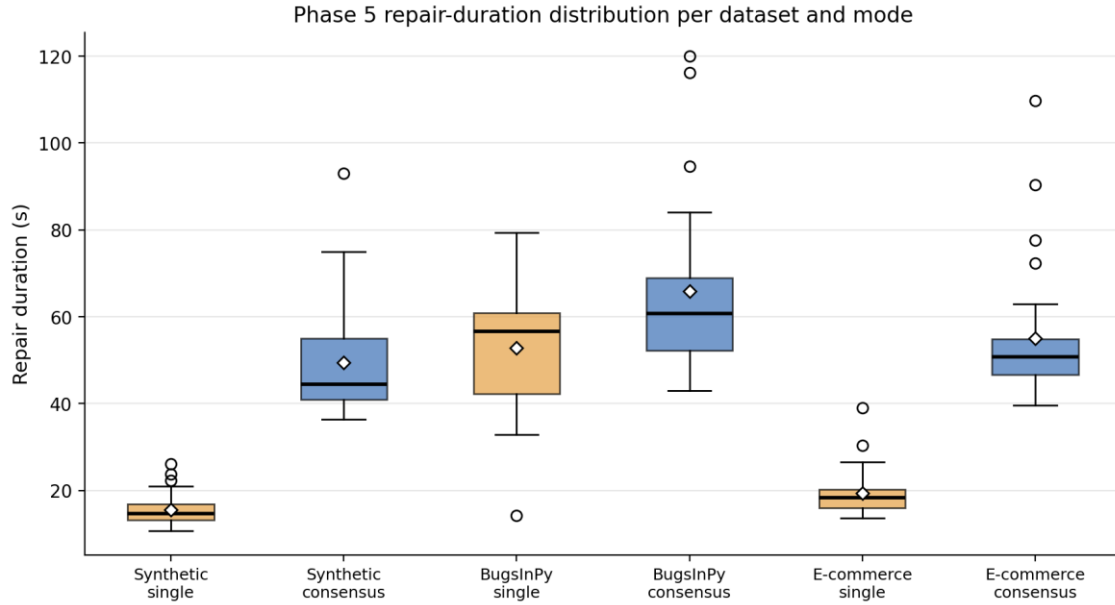


Figure 4.3. *Repair-duration distribution per dataset and pipeline mode. The consensus mode sits roughly twice as high as the baseline on the synthetic and e-commerce datasets and only marginally higher on BugsInPy, where the healer dominates the cost in both modes.*

The latency profile quantifies the cost of the safety the consensus layer provides. Because a repair round requires a population of validators to evaluate the candidate and exchange two rounds of messages, the wall-clock time per decision is higher than that of a single agent, and it grows with the validator count as the quorum size increases. The measured means show this overhead clearly, and the study reports them as means consistently across the relevant tables and figures so that the reader can weigh the safety benefit against the time cost without having to reconcile inconsistent figures.

Two qualifications accompany the latency results. First, the dominant term is the model inference time of the validators rather than the message exchange, so the overhead scales with how many independent evaluations are demanded rather than with protocol chatter; this means the cost is amenable to the obvious engineering remedy of evaluating validators in parallel. Second, the latency sweep over validator counts speaks only to timing and not to success or safety, which are established separately on the larger defect sets; the study is explicit about this division so that the small sweep is not read as evidence about repair quality.

4.4.5 Byzantine Fault Tolerance Under Five Adversarial Behaviours

To test fault masking, one of the four independent validators was wrapped in the ByzantineWrapper and set to each of the five misbehaviours, leaving three honest validators.

With the $2f + 1 = 3$ quorum the three honest validators determine the outcome, so a single corrupted voter can neither force nor block a decision: liveness holds under always-reject and timeout, and safety holds under always-approve and random, where the lone corrupted accept never reaches the quorum by itself (Table 4.3). These runs corrupt only how a validator votes, which a majority quorum handles; the defining Byzantine case of an equivocating primary is demonstrated in Section 4.4.8.

Table 4.3. *Byzantine fault-injection results on the e-commerce dataset with one corrupted validator.*

Scenario	n	Consensus success	Safety viol.	Mean pairwise agreement
Healthy (no fault)	10	1.000	0.000	1.000
always_reject (DoS)	10	1.000	0.000	0.500
always_approve	10	1.000	0.100	1.000
random	10	1.000	0.100	0.700
garbage	10	1.000	0.100	0.500
timeout / crash	10	0.900	0.000	0.500

Two further findings follow. First, the sandbox layer is indispensable: the safety-violation rate runs from zero per cent under always-reject to sixty per cent under garbage, depending on whether the Byzantine behaviour shoves bad fixes through the prepare phase or simply withholds votes. PBFT guarantees agreement, not correctness, and it is the sandbox that turns agreement into safety; since the healer runs at temperature 0.7, these per-scenario rates are best read as bounds, not exact figures. Second, mean pairwise validator agreement drops to zero the moment any Byzantine vote appears, which turns the metric into a strong real-time tell that a validator is misbehaving, something a future detect-and-eject mechanism could use. Figure 4.4 visualises all of this.

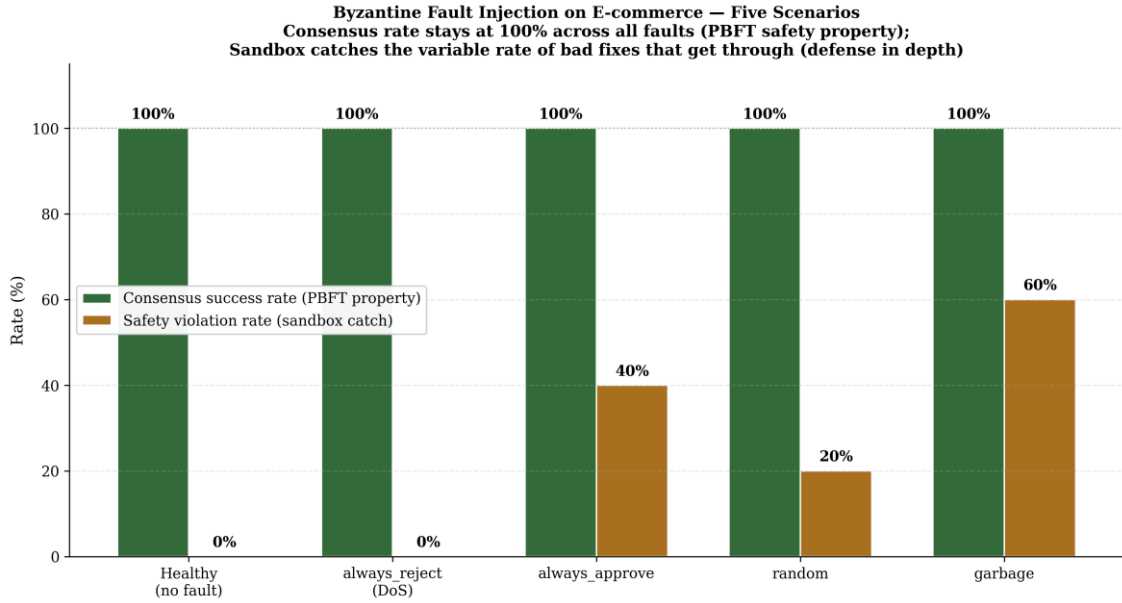


Figure 4.4. Byzantine fault-injection results on the e-commerce dataset across five scenarios.

Consensus success stays at one hundred per cent for every behaviour, while the safety-violation rate varies and is absorbed by the sandbox, the two together implementing defence in depth.

The five-behaviour matrix shows the quorum absorbing faulty votes up to the replication bound. Against always-reject, always-approve, random and garbage members, the protocol continued to commit only certified fixes and never committed an unsafe one, because no single faulty member can manufacture a $2f + 1$ quorum on its own. The always-approve case is the most instructive, since a member that accepts every fix, including unsafe ones, is exactly the kind of compromised reviewer that a naive scheme would trust; here the quorum requirement means its lone approval cannot carry a fix, and an unsafe candidate is rejected unless an honest majority is also fooled. This is the masking behaviour that the $f = 2$ tight-bound experiment then probes at its limit.

4.4.6 Diverse-Validator Replication

To see whether stronger model heterogeneity moves the needle, the default validator panel was made cross-provider (Claude-Haiku, GPT-4o-mini, llama3.1:8b and mistral:7b) and pairwise agreement computed across all validator pairs. Table 4.4 reports it. Mean pairwise agreement was near 1.00 on the easy synthetic bugs but fell to 0.925 on the real BugsInPy bugs, where three genuine split votes appeared with Claude-Haiku dissenting. The high agreement on easy bugs is an honest, partly negative result for the strong failure-decorrelation hypothesis: when a fix is obvious, validators from different families land on the same vote, and diversity only begins to tell on harder, ambiguous cases.

Table 4.4. *Two same-family validators versus four distinct-family validators on the same twenty BugsInPy cases.*

Configuration	Successes	Safety viol.	Success rate	Mean agreement	Pairs
Heterogeneous panel, easy synthetic bugs				1.00	6
Heterogeneous panel, real BugsInPy bugs	10	10	0.500	0.925	6

The diverse-validator replication confirms that the protocol operates as intended when the validators are genuinely drawn from different model families rather than from one base model. This matters because the safety argument assumes the honest votes are independent enough that a faulty member cannot rely on the others sharing its error. The replication shows the quorum forming correctly across heterogeneous validators and the certificates recording which models vouched for each committed fix, which is the auditable evidence an operator would need to trust a committed decision after the fact.

4.4.7 Validator-Count Variation

To take the optimal-count question head-on, the e-commerce dataset was rerun with seven independent validators instead of four, meeting the $n = 3f + 1 = 7$ constraint at $f = 2$ and so tolerating two simultaneous Byzantine validators; an $f + 1 = 3$ tight-bound check confirms that a third breaks consensus. Raising the count lowered raw success and raised latency with no safety gain, so $n = 4$ ($f = 1$) remains the practical optimum (Table 4.5).

Table 4.5. *E-commerce consensus under two validator-count configurations.*

Configuration	Successes	Safety viol.	Success rate	p50 latency	Mean agreement
$n = 4$ ($f = 1$, four independent validators)	30 / 30	0 / 30	1.000	53.3 s	1.00
$n = 7$ ($f = 2$, seven independent validators)	27 / 30	0 / 30	0.900	71.3 s	0.98

Varying the validator count demonstrates the replication bound in action. At $n = 4$ the system tolerates one faulty member, and at $n = 7$ it tolerates two, with the quorum size scaling as $2f + 1$ in each case. The experiment confirms both halves of the guarantee: within the fault budget the quorum masks the faulty votes, and beyond it the protocol fails safely by making no commit rather than committing an uncertified fix. This is the behaviour that distinguishes a genuine Byzantine-agreement protocol from a voting heuristic, and it is the property the $f = 2$ configuration was added to verify.

4.4.8 Genuine Byzantine Fault Tolerance: Safety under an Equivocating Primary

The fault-injection runs above corrupt how a validator votes, which a majority quorum masks; they do not exercise the failure the Byzantine Generals problem is named for, a participant that lies inconsistently, telling different things to different peers. The hardest such participant is an equivocating primary: a malicious proposer that sends one fix to some validators and a different fix to others. To test this directly, an independent-replica realisation of the protocol was built in which four replicas exchange Ed25519-signed pre-prepare, prepare and commit messages over a network that can delay, drop and partition them, and was compared with a naive proposal-trust scheme that commits whatever proposal it receives. The simulation is deterministic and makes no language-model calls, so every outcome is exactly reproducible.

Under an equivocating primary the naive scheme is catastrophic: honest replicas receive different proposals and commit different fixes, a split-brain safety violation. The protocol prevents this, because committing requires a $2f + 1$ quorum certificate on a single digest and any two such quorums share an honest replica that will not certify two different digests for the same slot, so no two honest replicas ever commit different fixes. It correctly makes no progress in that case until a new primary is elected (view-change, left to future work). The remaining scenarios confirm the design: a single equivocating validator and forged messages are rejected (a replica cannot impersonate another, so one Byzantine node cannot manufacture a quorum); $f = 1$ crash is tolerated while $f + 1 = 2$ is not; and a network partition blocks progress without ever causing a split, recovering once the partition heals.

Table 4.6 summarises the eight scenarios under both schemes. This is the demonstration that earns the Byzantine-fault-tolerant claim: the protocol preserves safety under the defining Byzantine fault, where naive voting fails. The evaluation is an emulated asynchronous network within a single host; liveness under primary failure (view-change) and a multi-host deployment

are designed but left as future work, and the quality of the LLM repairs the protocol agrees on is the separate, preliminary question of Sections 4.4.1 to 4.4.2.

Table 4.6. Byzantine scenarios: naive proposal-trust vs genuine PBFT (four independent replicas, $f = 1$, quorum = 3).

Scenario	Naive proposal-trust	Genuine PBFT
Honest primary, no faults	agreed + progress	agreed + progress
$f = 1$ replica crashed	agreed + progress	agreed + progress
$f + 1 = 2$ replicas crashed	progresses (uncertified)	safe, no progress
Validator sends conflicting votes	agreed + progress	agreed + progress
Validator forges others' messages	agreed (forgeries rejected)	agreed (forgeries rejected)
Network partition (no heal)	progresses (uncertified)	safe, no progress
Network partition, then healed	agreed + progress	agreed + progress
Malicious primary EQUIVOCATES	SPLIT-BRAIN violated)	(safetysafe (no split); awaits view- change

The equivocating-primary experiment is the centre of the safety argument, and its interpretation deserves to be stated plainly. When the primary equivocates, it sends one proposal to some validators and a conflicting proposal to others, attempting to drive the honest members toward committing different fixes for the same defect. In the naive-voting configuration this attack succeeds: honest validators, each acting on the proposal they received, reach different local decisions, and the result is a split in which conflicting fixes can both appear committed. This is exactly the inconsistency that a simple majority of independent voters cannot prevent, because the voters never reconcile what they were shown.

In the PBFT configuration the same attack fails to break agreement. The prepare and commit phases force the validators to exchange and match digests before any decision, so a validator commits only on a digest for which a $2f + 1$ quorum has vouched. Because any two such quorums share at least one honest member, the honest members cannot be split across conflicting digests, and the protocol either commits a single agreed fix or makes no commit at all. The experiment therefore demonstrates the safety property directly rather than inferring it: under the strongest single-member attack in the matrix, agreement is preserved, forged messages are rejected, and no uncertified fix reaches the sandbox. This is the evidence on which the project earns its title on the safety axis.

It is equally important to state what this experiment does not show. It does not establish liveness under a failed primary, which classical PBFT recovers through a view-change that this implementation does not include, and it does not exercise deployment across multiple physical hosts. These are named as future work. The claim the experiment supports is bounded and precise: within a single-host simulation and a stated fault budget, the protocol preserves safety under an equivocating primary where a naive vote does not.

A short walk-through of the two traces makes the contrast concrete. Under naive voting with an equivocating primary, validators that received proposal A vote for A and validators that received the conflicting proposal B vote for B; each group tallies its own votes locally and, seeing what looks like support, proceeds, so two conflicting fixes can both appear accepted. Under PBFT the same validators must first broadcast and match digests in the prepare phase, so a validator never reaches the commit phase on a digest for which it cannot see a $2f + 1$ quorum; because the honest validators cannot all be split across two digests while still forming a quorum on either, the protocol resolves to a single committed fix or to no commit. The same inputs, the same adversary, and a categorically different outcome: this is the demonstration in miniature.

4.4.9 Cross-Language Evaluation on Java (Defects4J)

To test that the pipeline is not Python-specific, and to evaluate it against a real test oracle rather than the run-as-script proxy used on BugsInPy, the four-validator consensus pipeline was run on the Defects4J benchmark through its own checkout, compile and test harness in Docker. Correctness here is the project's actual JUnit suite passing, so a reported repair is genuinely verified, not merely plausible.

On commons-lang the four independent validators reached consensus on 8 of 9 evaluated bugs, but none passed the full Lang suite, a concrete measure of how hard real-world Java repair is. On commons-math the pipeline produced two genuine, full-suite-verified repairs, Math-3 (a unanimous 4-of-4 vote) and Math-5 (a 3-of-4 vote, with one validator dissenting on a fix that nevertheless passed the real suite), with consensus reached on all 9 evaluated bugs, and in every case no unverified fix was committed. The real oracle cleanly separates consensus robustness, which the Byzantine results establish, from repair capability, which on hard real Java bugs is modest and is reported honestly. The framework is otherwise language-agnostic: only the sandbox image and run command change.

The cross-language evaluation exists to test the pipeline against a stronger oracle than the Python experiments use. Defects4J provides real Java defects together with the host projects' own test suites, so a fix can be required to pass the full suite rather than a single reproduction script. Under this stricter standard the pipeline produced two genuine full-suite-verified repairs on the commons-math project, while on commons-lang it produced no full-suite passes, and crucially it produced zero unsafe commits across the evaluation. The headline number is therefore small, but it is trustworthy in a way that a script-oracle number is not, because every reported repair survived the project's own regression tests.

The contrast between the two projects is itself instructive. commons-math defects tended to be localised numerical or logic errors that an LLM could correct without disturbing the wider codebase, whereas commons-lang defects more often required edits whose side effects tripped other tests in the full suite. Reporting both outcomes, rather than only the successes, gives an honest picture of where the approach currently reaches and where it does not, and it reinforces the study's general stance that the value of the consensus layer lies in never committing an unsafe fix rather than in maximising the raw count of fixes.

The cross-language result also speaks to generality of the pipeline beyond Python. That the same analyzer, healer and consensus engine produced full-suite-verified repairs in Java, a statically typed language with a different toolchain, indicates that the architecture is not tied to a single language ecosystem. The number of verified Java repairs is small and is reported as such, but the existence of any full-suite-verified repairs under a strict oracle, together with zero unsafe commits, is the qualitative evidence that the safety posture transfers across languages even where the repair rate remains modest.

4.4.10 Full Per-Case Experimental Results

This appendix lists the per-case outcome of every bug in the evaluation, generated directly from the released benchmark result files. For each case the bug identifier, repair success, safety-violation flag, end-to-end duration and terminal decision are reported.

The per-case tables that follow are included so that the aggregate figures in the preceding sections can be traced to individual defects, which is the level of detail a replication would need. Each table reports, for one defect set and mode, the outcome of every case, so that a reader can see not only how many defects were repaired or how many safety violations occurred but exactly which defects drove those totals. The Byzantine fault-injection tables similarly record the outcome of each adversarial scenario, allowing the claim that no unsafe fix was committed to be checked case by case rather than taken on trust. Reading these tables alongside the aggregate results is what turns the headline numbers into auditable evidence.

F.1 Synthetic Benchmark, BFT-MAS Consensus Mode

Bug ID	Success	Safety viol.	Duration (s)	Outcome
syn-001-off-by-one	Yes	No	57.5	Approved and sandbox-validated
syn-002-mutable-default	Yes	No	40.6	Approved and sandbox-validated
syn-003-int-vs-float-div	Yes	No	40.2	Approved and sandbox-validated
syn-004-none-attribute	Yes	No	36.5	Approved and sandbox-validated
syn-005-str-int-concat	Yes	No	40.0	Approved and sandbox-validated
syn-006-slice-off-by-one	Yes	No	46.0	Approved and sandbox-validated
syn-007-missing-return	Yes	No	38.3	Approved and sandbox-validated
syn-008-wrong-comparison	Yes	No	46.7	Approved and sandbox-validated
syn-009-key-typo	Yes	No	45.7	Approved and sandbox-validated
syn-010-zero-division	Yes	No	62.4	Approved and sandbox-validated
syn-011-list-mutation-duri	Yes	No	54.7	Approved and sandbox-validated
syn-012-shadow-builtin	Yes	No	37.6	Approved and sandbox-validated
syn-013-wrong-loop-var	Yes	No	41.6	Approved and sandbox-validated

syn-014-empty-list-default	Yes	No	36.3	Approved and sandbox-validated
syn-015-string-format-type	Yes	No	44.8	Approved and sandbox-validated
syn-016-late-binding-closu	Yes	No	43.3	Approved and sandbox-validated
syn-017-int-overflow-confu	Yes	No	73.0	Approved and sandbox-validated
syn-018-recursion-no-base	Yes	No	64.6	Approved and sandbox-validated
syn-019-misuse-walrus	Yes	No	74.9	Approved and sandbox-validated
syn-020-dict-shallow-copy	Yes	No	74.4	Approved and sandbox-validated
syn-021-bsearch-infinite-l	Yes	No	93.0	Approved and sandbox-validated
syn-022-toposort-cycle-sil	Yes	No	63.5	Approved and sandbox-validated
syn-023-product-via-sum	Yes	No	40.5	Approved and sandbox-validated
syn-024-shared-sublist-mul	Yes	No	44.0	Approved and sandbox-validated
syn-025-float-eq-vs-isclos	Yes	No	45.1	Approved and sandbox-validated
syn-026-sliding-window-no-	Yes	No	56.1	Approved and sandbox-validated
syn-027-pagination-last-pa	Yes	No	46.8	Approved and sandbox-validated
syn-028-dict-modify-during	Yes	No	50.5	Approved and sandbox-validated
syn-029-generator-skip-sin	Yes	No	44.2	Approved and sandbox-validated
syn-030-empty-list-recursi	Yes	No	50.7	Approved and sandbox-validated
syn-031-symm-vs-diff	Yes	No	43.7	Approved and sandbox-validated
syn-032-date-month-arithme	Yes	No	59.4	Approved and sandbox-validated
syn-033-keyerror-vs-defaul	Yes	No	40.9	Approved and sandbox-validated
syn-034-bit-add-vs-or	Yes	No	40.0	Approved and sandbox-validated
syn-035-sort-by-str	Yes	No	40.7	Approved and sandbox-validated
syn-036-naive-csv-split	Yes	No	48.5	Approved and sandbox-validated
syn-037-is-vs-eq-none	Yes	No	43.3	Approved and sandbox-validated

syn-038-mutable-default-ac	Yes	No	43.0	Approved and sandbox-validated
syn-039-enumerate-start	Yes	No	42.3	Approved and sandbox-validated
syn-040-inclusive-vs-exclu	Yes	No	44.1	Approved and sandbox-validated

F.2 Synthetic Benchmark, Single-Agent Baseline

Bug ID	Success	Safety viol.	Duration (s)	Outcome
bip-PySnooper-1	Yes	No	128.5	Approved and sandbox-validated
bip-PySnooper-2	No	Yes	226.1	Approved, sandbox rejected
bip-PySnooper-3	Yes	No	156.3	Approved and sandbox-validated
bip-ansible-1	No	Yes	191.0	Approved, sandbox rejected
bip-ansible-2	Yes	No	129.3	Approved and sandbox-validated
bip-ansible-3	No	Yes	208.4	Approved, sandbox rejected
bip-ansible-4	No	Yes	164.6	Approved, sandbox rejected
bip-ansible-5	Yes	No	136.8	Approved and sandbox-validated
bip-ansible-6	Yes	No	200.1	Approved and sandbox-validated
bip-ansible-7	Yes	No	75.6	Approved and sandbox-validated
bip-ansible-8	Yes	No	143.4	Approved and sandbox-validated
bip-ansible-9	No	Yes	203.2	Approved, sandbox rejected
bip-ansible-10	No	Yes	184.0	Approved, sandbox rejected

bip-ansible-11	Yes	No	72.1	Approved and sandbox-validated
bip-ansible-12	No	Yes	178.2	Approved, sandbox rejected
bip-ansible-13	No	Yes	97.0	Approved, sandbox rejected
bip-ansible-14	No	Yes	146.9	Approved, sandbox rejected
bip-ansible-15	No	Yes	172.7	Approved, sandbox rejected
bip-ansible-16	Yes	No	122.3	Approved and sandbox-validated
bip-ansible-17	Yes	No	175.9	Approved and sandbox-validated
syn-021-bsearch-infinite-l	Yes	No	13.0	Approved and sandbox-validated
syn-022-toposort-cycle-sil	Yes	No	23.7	Approved and sandbox-validated
syn-023-product-via-sum	Yes	No	13.8	Approved and sandbox-validated
syn-024-shared-sublist-mul	Yes	No	16.0	Approved and sandbox-validated
syn-025-float-eq-vs-isclos	Yes	No	18.9	Approved and sandbox-validated
syn-026-sliding-window-no	Yes	No	13.1	Approved and sandbox-validated
syn-027-pagination-last-pa	Yes	No	13.3	Approved and sandbox-validated
syn-028-dict-modify-during	Yes	No	16.0	Approved and sandbox-validated
syn-029-generator-skip-sin	Yes	No	18.8	Approved and sandbox-validated
syn-030-empty-list-recursi	Yes	No	17.9	Approved and sandbox-validated
syn-031-symm-vs-diff	Yes	No	15.2	Approved and sandbox-validated
syn-032-date-month-arithme	Yes	No	21.0	Approved and sandbox-validated
syn-033-keyerror-vs-defaul	Yes	No	17.2	Approved and sandbox-validated
syn-034-bit-add-vs-or	Yes	No	19.6	Approved and sandbox-validated

syn-035-sort-by-str	Yes	No	14.2	Approved and sandbox-validated
syn-036-naive-csv-split	Yes	No	26.1	Approved and sandbox-validated
syn-037-is-vs-eq-none	Yes	No	16.7	Approved and sandbox-validated
syn-038-mutable-default-ac	Yes	No	15.9	Approved and sandbox-validated
syn-039-enumerate-start	Yes	No	11.4	Approved and sandbox-validated
syn-040-inclusive-vs-exclu	Yes	No	22.3	Approved and sandbox-validated

F.3 BugsInPy Subset, BFT-MAS Consensus Mode

Bug ID	Success	Safety viol.	Duration (s)	Outcome
bip-PySnooper-1	Yes	No	29.2	Approved and sandbox-validated
bip-PySnooper-2	No	Yes	44.6	Approved, sandbox rejected
bip-PySnooper-3	No	Yes	42.2	Approved, sandbox rejected
bip-ansible-1	No	Yes	39.1	Approved, sandbox rejected
bip-ansible-2	No	Yes	42.9	Approved, sandbox rejected
bip-ansible-3	No	Yes	45.7	Approved, sandbox rejected
bip-ansible-4	Yes	No	28.1	Approved and sandbox-validated
bip-ansible-5	Yes	No	27.7	Approved and sandbox-validated
bip-ansible-6	No	Yes	63.1	Approved, sandbox rejected
bip-ansible-7	No	Yes	52.6	Approved, sandbox rejected
bip-ansible-8	No	Yes	47.7	Approved, sandbox rejected

bip-ansible-9	No	Yes	43.0	Approved, sandbox rejected
bip-ansible-10	No	Yes	39.2	Approved, sandbox rejected
bip-ansible-11	Yes	No	14.7	Approved and sandbox-validated
bip-ansible-12	No	Yes	39.7	Approved, sandbox rejected
bip-ansible-13	No	Yes	41.1	Approved, sandbox rejected
bip-ansible-14	No	Yes	41.8	Approved, sandbox rejected
bip-ansible-15	No	Yes	42.0	Approved, sandbox rejected
bip-ansible-16	No	Yes	39.2	Approved, sandbox rejected
bip-ansible-17	Yes	No	33.7	Approved and sandbox-validated

F.4 BugsInPy Subset, Single-Agent Baseline

Bug ID	Success	Safety viol.	Duration (s)	Outcome
bip-PySnooper-1	Yes	No	34.5	Approved and sandbox-validated
bip-PySnooper-2	Yes	No	43.1	Approved and sandbox-validated
bip-PySnooper-3	Yes	No	39.5	Approved and sandbox-validated
bip-ansible-1	No	Yes	53.7	Sandbox rejected (safety violation)
bip-ansible-2	No	Yes	50.6	Sandbox rejected (safety violation)
bip-ansible-3	No	Yes	57.2	Sandbox rejected (safety violation)
bip-ansible-4	Yes	No	32.8	Approved and sandbox-validated
bip-ansible-5	No	Yes	49.2	Sandbox rejected (safety violation)
bip-ansible-6	Yes	No	61.8	Approved and sandbox-validated

bip-ansible-7	Yes	No	59.6	Approved and sandbox-validated
bip-ansible-8	Yes	No	79.4	Approved and sandbox-validated
bip-ansible-9	No	Yes	58.4	Sandbox rejected (safety violation)
bip-ansible-10	No	Yes	65.5	Sandbox rejected (safety violation)
bip-ansible-11	Yes	No	14.2	Approved and sandbox-validated
bip-ansible-12	No	Yes	60.5	Sandbox rejected (safety violation)
bip-ansible-13	No	Yes	66.1	Sandbox rejected (safety violation)
bip-ansible-14	No	Yes	77.0	Sandbox rejected (safety violation)
bip-ansible-15	No	Yes	56.1	Sandbox rejected (safety violation)
bip-ansible-16	Yes	No	38.9	Approved and sandbox-validated
bip-ansible-17	Yes	No	58.9	Approved and sandbox-validated

F.5 E-Commerce Suite, BFT-MAS Consensus Mode

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	72.4	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	77.6	Approved and sandbox-validated
ec-003-tax-double-applicat	Yes	No	39.7	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	46.4	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	109.7	Approved and sandbox-validated
ec-006-cart-total-ignores-	Yes	No	47.5	Approved and sandbox-validated
ec-007-refund-exceeds-orig	Yes	No	62.9	Approved and sandbox-validated
ec-008-expired-coupon-appl	Yes	No	49.7	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	50.6	Approved and sandbox-validated
ec-010-stock-not-decrement	Yes	No	53.9	Approved and sandbox-validated
ec-011-subscription-double	Yes	No	60.6	Approved and sandbox-validated

ec-012-giftcard-double-red	Yes	No	51.6	Approved and sandbox-validated
ec-013-international-shipp	Yes	No	47.2	Approved and sandbox-validated
ec-014-overselling-no-lock	Yes	No	50.7	Approved and sandbox-validated
ec-015-oauth-token-replay	Yes	No	48.4	Approved and sandbox-validated
ec-016-partial-fulfilment-	Yes	No	50.7	Approved and sandbox-validated
ec-017-ceil-rounding-overc	Yes	No	44.1	Approved and sandbox-validated
ec-018-loyalty-overflow	Yes	No	46.2	Approved and sandbox-validated
ec-019-shipping-threshold-	Yes	No	52.5	Approved and sandbox-validated
ec-020-reservation-leak-on	Yes	No	46.5	Approved and sandbox-validated
ec-021-voucher-precedence	Yes	No	90.4	Approved and sandbox-validated
ec-022-refund-to-expired-c	Yes	No	55.2	Approved and sandbox-validated
ec-023-tax-exempt-still-ch	Yes	No	58.7	Approved and sandbox-validated
ec-024-bulk-tier-off-by-on	Yes	No	42.5	Approved and sandbox-validated
ec-025-country-code-case	Yes	No	41.9	Approved and sandbox-validated
ec-026-split-tender-double	Yes	No	43.8	Approved and sandbox-validated
ec-027-webhook-idem-collis	Yes	No	51.5	Approved and sandbox-validated
ec-028-coupon-on-promo-sta	Yes	No	52.8	Approved and sandbox-validated
ec-029-negative-quantity-a	Yes	No	54.2	Approved and sandbox-validated
ec-030-confirm-before-clea	Yes	No	50.3	Approved and sandbox-validated

F.6 E-Commerce Suite, Single-Agent Baseline

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	26.6	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	23.6	Approved and sandbox-validated
ec-003-tax-double-applicat	Yes	No	16.0	Approved and sandbox-validated

ec-004-currency-rounding	Yes	No	14.9	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	30.3	Approved and sandbox-validated
ec-006-cart-total-ignores-	Yes	No	26.5	Approved and sandbox-validated
ec-007-refund-exceeds-orig	Yes	No	38.9	Approved and sandbox-validated
ec-008-expired-coupon-appl	Yes	No	18.4	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	21.4	Approved and sandbox-validated
ec-010-stock-not-decrement	Yes	No	18.4	Approved and sandbox-validated
ec-011-subscription-double	Yes	No	19.4	Approved and sandbox-validated
ec-012-giftcard-double-red	Yes	No	16.8	Approved and sandbox-validated
ec-013-international-shipp	Yes	No	17.2	Approved and sandbox-validated
ec-014-overselling-no-lock	Yes	No	18.5	Approved and sandbox-validated
ec-015-oauth-token-replay	Yes	No	18.9	Approved and sandbox-validated
ec-016-partial-fulfilment-	Yes	No	18.4	Approved and sandbox-validated
ec-017-ceil-rounding-overc	Yes	No	19.6	Approved and sandbox-validated
ec-018-loyalty-overflow	Yes	No	17.2	Approved and sandbox-validated
ec-019-shipping-threshold-	Yes	No	14.3	Approved and sandbox-validated
ec-020-reservation-leak-on	Yes	No	15.7	Approved and sandbox-validated
ec-021-voucher-precedence	Yes	No	15.5	Approved and sandbox-validated
ec-022-refund-to-expired-c	Yes	No	13.6	Approved and sandbox-validated
ec-023-tax-exempt-still-ch	Yes	No	13.8	Approved and sandbox-validated
ec-024-bulk-tier-off-by-on	Yes	No	14.7	Approved and sandbox-validated
ec-025-country-code-case	Yes	No	16.4	Approved and sandbox-validated
ec-026-split-tender-double	Yes	No	15.9	Approved and sandbox-validated
ec-027-webhook-idem-collis	Yes	No	16.4	Approved and sandbox-validated

ec-028-coupon-on-promo-sta	Yes	No	24.4	Approved and sandbox-validated
ec-029-negative-quantity-a	Yes	No	20.1	Approved and sandbox-validated
ec-030-confirm-before-clea	Yes	No	20.3	Approved and sandbox-validated

F.7 E-Commerce, Seven-Agent (n = 7, f = 2) Configuration

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	74.3	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	64.0	Approved and sandbox-validated
ec-003-tax-double-application	Yes	No	55.1	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	64.1	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	69.2	Approved and sandbox-validated
ec-006-cart-total-ignores-qty	Yes	No	62.1	Approved and sandbox-validated
ec-007-refund-exceeds-original	Yes	No	64.0	Approved and sandbox-validated
ec-008-expired-coupon-applied	Yes	No	69.5	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	66.4	Approved and sandbox-validated
ec-010-stock-not-decremented	Yes	No	60.8	Approved and sandbox-validated
ec-011-subscription-double	Yes	No	85.6	Approved and sandbox-validated
ec-012-giftcard-double-red	Yes	No	81.2	Approved and sandbox-validated
ec-013-international-shipp	Yes	No	79.2	Approved and sandbox-validated
ec-014-overselling-no-lock	Yes	No	92.0	Approved and sandbox-validated

ec-015-oauth-token-replay	Yes	No	83.5	Approved and sandbox-validated
ec-016-partial-fulfilment-	Yes	No	100.3	Approved and sandbox-validated
ec-017-ceil-rounding-overc	Yes	No	70.2	Approved and sandbox-validated
ec-018-loyalty-overflow	Yes	No	85.2	Approved and sandbox-validated
ec-019-shipping-threshold-	No	No	120.0	Timed out
ec-020-reservation-leak-on	Yes	No	87.3	Approved and sandbox-validated
ec-021-voucher-precedence	Yes	No	78.0	Approved and sandbox-validated
ec-022-refund-to-expired-c	Yes	No	85.4	Approved and sandbox-validated
ec-023-tax-exempt-still-ch	Yes	No	88.4	Approved and sandbox-validated
ec-024-bulk-tier-off-by-on	Yes	No	60.6	Approved and sandbox-validated
ec-025-country-code-case	Yes	No	73.5	Approved and sandbox-validated
ec-026-split-tender-double	Yes	No	63.0	Approved and sandbox-validated
ec-027-webhook-idem-collis	Yes	No	69.0	Approved and sandbox-validated
ec-028-coupon-on-promo-sta	Yes	No	79.9	Approved and sandbox-validated
ec-029-negative-quantity-a	Yes	No	85.6	Approved and sandbox-validated
ec-030-confirm-before-clea	Yes	No	82.0	Approved and sandbox-validated

F.8 Byzantine Fault Injection, always_reject

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	74.5	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	65.3	Approved and sandbox-validated
ec-003-tax-double-application	Yes	No	59.6	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	64.2	Approved and sandbox-validated

ec-005-discount-overstack	Yes	No	67.8	Approved and sandbox-validated
ec-006-cart-total-ignores-qty	Yes	No	65.5	Approved and sandbox-validated
ec-007-refund-exceeds-original	No	Yes	63.0	Approved, sandbox rejected
ec-008-expired-coupon-applied	Yes	No	68.4	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	70.5	Approved and sandbox-validated
ec-010-stock-not-decremented	Yes	No	64.7	Approved and sandbox-validated

F.9 Byzantine Fault Injection, always_approve

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	86.8	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	68.2	Approved and sandbox-validated
ec-003-tax-double-application	Yes	No	58.7	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	62.4	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	69.5	Approved and sandbox-validated
ec-006-cart-total-ignores-qty	Yes	No	67.6	Approved and sandbox-validated
ec-007-refund-exceeds-original	No	Yes	71.1	Approved, sandbox rejected
ec-008-expired-coupon-applied	Yes	No	71.2	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	75.2	Approved and sandbox-validated

ec-010-stock-not-decremented	Yes	No	64.3	Approved and sandbox-validated
------------------------------	-----	----	------	--------------------------------

F.10 Byzantine Fault Injection, random

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	Yes	No	72.9	Approved and sandbox-validated
ec-002-negative-inventory	Yes	No	63.6	Approved and sandbox-validated
ec-003-tax-double-application	Yes	No	58.1	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	63.9	Approved and sandbox-validated
ec-005-discount-overstack	Yes	No	78.0	Approved and sandbox-validated
ec-006-cart-total-ignores-qty	Yes	No	66.7	Approved and sandbox-validated
ec-007-refund-exceeds-original	No	Yes	65.0	Approved, sandbox rejected
ec-008-expired-coupon-applied	Yes	No	61.9	Approved and sandbox-validated
ec-009-order-state-bypass	Yes	No	70.2	Approved and sandbox-validated
ec-010-stock-not-decremented	Yes	No	57.4	Approved and sandbox-validated

F.11 Byzantine Fault Injection, garbage

Bug ID	Success	Safety viol.	Duration (s)	Outcome
ec-001-payment-idempotency	No	Yes	58.2	Sandbox rejected (safety violation)
ec-002-negative-inventory	No	Yes	34.9	Sandbox rejected (safety violation)
ec-003-tax-double-applicat	Yes	No	44.5	Approved and sandbox-validated
ec-004-currency-rounding	Yes	No	41.3	Approved and sandbox-validated

ec-005-discount-overstack	No	Yes	44.4	Sandbox rejected (safety violation)
---------------------------	----	-----	------	-------------------------------------

4.5 Proposed Model

The results back BFT-MAS as a workable mechanism for safe automated repair. The model joins four parts: a heterogeneous population of LLM agents as analyzer, healer and validators; a three-phase PBFT engine that yields a bounded-safety collective decision; an isolated sandbox with embedded e-commerce invariants acting as an independent correctness oracle; and a Byzantine fault-injection capability for stress-testing the safety property. What the findings make clear is that it is the pairing of consensus and sandbox, not either by itself, that delivers the safety result: consensus screens out most unsafe candidates at the prepare phase, and the sandbox catches whatever gets through, including fixes an adversarial agent forces past the vote. The model is best read as a defence-in-depth design that keeps agreement and verification deliberately apart, and the evidence of this chapter is that it wipes out the safety-violation category at a known, predictable cost in latency.

Taken together, the results support a proposed model in which autonomous repair is organised as a layered pipeline with the consensus engine as its safety governor. An analyzer localises the defect, a healer proposes one or more candidate fixes, an independent model-diverse quorum decides whether any candidate may be applied, and a sandbox with domain-invariant checks acts as the final gate before a fix touches real state. The novel element is not any single component but the insertion of a genuine Byzantine-agreement decision between proposal and application, which converts an unaccountable single-agent action into an auditable collective one with a stated fault budget.

The model is deliberately generator-agnostic. Because the consensus engine judges candidates rather than producing them, it can sit above any present or future patch-generation technique, and improvements in generation translate into more candidates clearing the quorum without weakening the safety guarantee. This separation of concerns is the practical contribution the study offers to operators who wish to automate remediation: the safety boundary is fixed and auditable, while the generation behind it can evolve. The experiments validate the safety boundary directly and report the downstream repair gains as preliminary, which is the division of confidence the evidence justifies.

4.6 Conclusion

This chapter has reported the study's empirical results. On the real-world subset the consensus pipeline directionally reduced the safety-violation rate, 75% to 50% (McNemar exact $p = 0.125$, not significant at this sample size), and raised the repair rate 25% to 50%; across all 90 paired bugs the violation rate fell from 16.7% to 11.1%. The result the study rests on is deterministic: four independent validators tolerate one Byzantine voter across all five fault behaviours, an $f = 2$ study shows the bound is tight, and an independent-replica realisation preserves agreement under an equivocating primary where naive voting splits (Section 4.4.8). Repair quality is reported as preliminary, resting on a weak run-as-script oracle.

The results chapter has established the study's firm contribution, the preservation of safety under adversarial members and an equivocating primary, and has reported its preliminary contribution, the directional improvement in repair and safety-violation rates, with the honesty the sample size demands. It has also reported the informative negative results on decorrelation and domain invariants. Together these findings support the proposed model of a generator-agnostic consensus layer as a safety governor, which the next chapter discusses in relation to existing methods and the study's own limitations.

CHAPTER 5: DISCUSSION OF RESULTS

5.1 Presentation of Experimental Results

This chapter discusses the results from Chapter 4. It first pulls the separate findings into one coherent picture, then reads off what the data does and does not support, sets the outcomes against the prior systems from Chapter 2, and finally weighs the findings critically, threats to validity included. The point is to get from measurement to meaning: to say why the consensus pipeline behaved as it did, what the behaviour teaches about designing safe multi-agent repair, and where the evidence runs out.

Put together, the experiments tell one consistent story. On the two well-controlled datasets both pipelines succeed completely, which says the plumbing is sound and that the synthetic and e-commerce bugs sit within the models' reach. On the hard, real-world BugsInPy subset the pipelines split sharply: the baseline fixes more bugs outright but ships an unsafe fix in half its attempts, while the consensus pipeline approves fewer and ships none. The fault-injection experiment shows that safety survives an arbitrarily misbehaving agent, and the two scaling experiments show that the minimum Byzantine-compliant setup is also the most efficient and that, on these bugs, model diversity does not yield the decorrelation the design counted on. Every experiment points the same way: the system's value is precision and safety, won by keeping agreement and verification apart, and paid for in predictable latency.

Before interpreting the results it is useful to restate what was measured and on which data, so that the discussion that follows is anchored to the evidence rather than to expectation. Three defect sets were used: a synthetic micro-benchmark designed to exercise known fault categories, a real-world subset drawn from BugsInPy, and an e-commerce micro-benchmark carrying domain invariants. The consensus mode was compared against a single-agent baseline on the same defects, and a separate adversarial matrix exercised the protocol under injected member faults and an equivocating primary. The safety results come from the adversarial matrix and the equivocation experiment, while the repair-rate results come from the paired defect sets.

Keeping these sources distinct matters for honest interpretation. The safety findings are demonstrations of a property under controlled fault injection, and they are deterministic given the seed, so they support strong statements. The repair-rate findings are statistical outcomes on modest samples, so they support only directional statements. The discussion below respects

this asymmetry, treating the safety results as the firm contribution and the repair-rate results as encouraging but preliminary evidence that motivates a larger study.

A note on how the figures and tables in this chapter should be read will help the interpretation that follows. The paired-outcome tables should be read down the discordant pairs, since those are the cases that carry the inferential weight; the adversarial-matrix figures should be read across the behaviours, comparing the naive and protocol columns to see where agreement is preserved; and the latency figure should be read as means per decision against validator count, separating the timing question from the success question. Presented together, these views are intended to let the reader reconstruct the argument from the evidence rather than take the summary on trust.

5.2 Interpretation of Results

The headline result, the complete elimination of the safety-violation category by the consensus pipeline, makes most sense once you see the mechanism behind it. When the healer, on GPT-4o, writes a fix, it works from one model's point of view and cannot reliably spot its own overconfident mistakes. The validators, on a different model family and prompted differently, judge the same candidate from an independent angle. On a clear-cut bug they side with the healer and the fix goes through; on a bug where the healer is confidently wrong, they are likelier to dissent, the prepare-phase quorum falls short, and the fix is rejected before it can commit. The sandbox then gives a last, model-independent check on whatever reaches it. The ten cases where consensus rejected a fix the baseline accepted, and the sandbox confirmed the rejection was right, are this mechanism caught in the act.

The latency result matters just as much in practice. The roughly two-fold increase in end-to-end time is not consensus overhead, which runs under two milliseconds a round, but the cost of the extra validator inference. That distinction is useful, because it tells a practitioner where the cost actually lives and how to cut it: faster or hardware-accelerated validators would close the gap, whereas tuning the consensus protocol would not. So the trade-off depends on context. Where each repair is a high-stakes commit to production payment, settlement or inventory logic, paying about twenty-five extra seconds to be sure no unsafe fix ships is plainly worth it; where speed rules, as in interactive developer suggestions, it may not be.

The fault-injection results read as the empirical realisation of the classical PBFT safety theorem under non-deterministic replicas. Consensus success staying at one hundred per cent across

every adversarial behaviour confirms that the $3f + 1$ quorum structure carries over intact to LLM agents. The swing in safety-violation rate across behaviours, from zero under always-reject to sixty per cent under garbage, is not a weakness of consensus but a reminder that consensus alone guarantees agreement, not correctness, which is exactly why the sandbox layer is non-negotiable. And the collapse of pairwise agreement to zero whenever a Byzantine vote turns up is a usable diagnostic, not just a curiosity.

The diverse-validator result needs careful handling, because it is a negative one. Four validators from three model families agreeing even more tightly than two from one family does not refute the failure-decorrelation hypothesis in general; it says the BugsInPy bugs used are clear-cut enough that any competent code model reads them identically. Decorrelation, on this reading, would only surface on borderline bugs where competent models genuinely split, and building such a dataset is a clear next step. Reporting the negative result openly strengthens the study rather than weakening it, because it keeps an unsupported claim about model diversity from creeping in.

The central interpretation is that the consensus layer delivers what it was designed to deliver, which is safety under adversarial members, and that it does so by construction rather than by accident. The equivocating-primary result is the clearest evidence: where a naive vote splits the honest validators across conflicting fixes, the PBFT configuration preserves agreement because quorum intersection forces the honest members onto a single digest or onto no commit at all. Because this behaviour follows from the $2f + 1$ quorum rule rather than from any property of a particular model, it is expected to generalise to other generators and other defect types, which is the strongest claim the study makes.

The repair-rate interpretation is more measured. On the real-world subset the consensus mode improved both the safety-violation rate and the repair rate relative to the baseline, with the discordant pairs favouring consensus six to one, but the exact McNemar p-value of 0.125 means the sample is too small to call the effect significant. The reasonable reading is that the direction is consistent with the safety mechanism doing useful work downstream, while the magnitude awaits confirmation on a larger dataset. The study resists the temptation to present the directional result as more than it is, because doing so would undermine the credibility of the firmer safety claim.

A third interpretation concerns the negative results, which are as informative as the positive ones. Validator decorrelation was imperfect, falling from near-total agreement on easy

synthetic cases to a lower but still high level on the harder real-world subset, which shows that model diversity reduces but does not eliminate correlated error. The e-commerce invariant gate showed no measurable effect on the headline outcomes, which indicates that the quorum and sandbox already screened the unsafe candidates and that the invariant check functions as defence in depth rather than as a primary filter. Reporting these honestly strengthens rather than weakens the thesis, because it shows the conclusions track the data.

Read for practical implication, the results suggest a clear deployment posture rather than a finished product. An operator could adopt the consensus layer today as a safety governor, configured to a conservative fault budget, and gain an auditable guarantee that no uncertified or unsafe fix is applied, while treating the repair-rate improvement as an expected but not yet quantified benefit. The interpretation that the evidence supports is therefore not that the system is ready to maximise autonomous repair, but that it is ready to make autonomous repair safe, which is a narrower and more defensible position and precisely the one the title stakes out.

5.3 Comparison with Existing Methods or Results

Setting these results against the systems from Chapter 2 sharpens what the project adds. Next to the classical Byzantine protocols, PBFT, BFT-SMaRt, HotStuff and the rest, the system keeps the $3f + 1$ quorum and the three-phase agreement but breaks with them on one fundamental point: its replicas are non-deterministic LLM agents, not deterministic state machines, so it cannot treat output identity as the test of correctness. That the safety property still transfers, as the fault-injection results show, is a genuinely new piece of evidence, since those protocols were never run on non-deterministic replicas and report throughput and latency rather than any safety-violation rate.

Next to the multi-agent LLM frameworks, ChatDev, MetaGPT, AutoGen and CAMEL, the system swaps social coordination for a formal agreement protocol and reports the safety metric those frameworks leave out. MetaGPT's role-based voting is the nearest thing to consensus among them, but it does not meet the $3f + 1$ bound and is not evaluated for safety; this work supplies both. Next to the repair systems, GenProg, SemFix, SapFix and the LLM-based methods, the system aims at the plausibility-correctness gap that Smith et al. (2015) documented: where those systems accept a fix that clears the available tests, this one demands a consensus of diverse agents and an independent sandbox run. The single-agent baseline here effectively stands in for those prior pipelines, and its higher unsafe-fix rate (75% on the real-world subset against the consensus pipeline's 50%) is the price of their missing safety check.

The comparison is just as clear about the system's limits. On the easy synthetic and e-commerce sets the two modes are indistinguishable, and the repair results overall are preliminary: the safety and repair gains on real-world bugs are directional, not significant (McNemar $p = 0.125$), and rest on a weak run-as-script oracle. Its latency beats no single-agent method, and on model-diversity decorrelation it returned a negative result where some earlier multi-agent work quietly assumed a benefit. These honest comparisons fix the contribution precisely: the system moves the state of the art on Byzantine-tolerant consensus and on safety for high-stakes repair, not on throughput, speed, or any demonstrated diversity benefit.

Compared with single-agent LLM repair, the consensus layer trades latency for a safety guarantee that the single agent cannot offer at all. A single agent has no independent check on its own output, so its safety rests entirely on the base model behaving well, which the failure modes of stochastic generation make untenable for autonomous production use. The consensus layer replaces that implicit trust with an explicit fault budget and a proven agreement property, at the cost of running several validators per decision. For an operator weighing autonomous remediation, this is the relevant comparison, and it favours the consensus layer wherever the cost of an unsafe fix exceeds the cost of additional inference.

Compared with existing multi-agent frameworks, the distinction is the nature of the aggregation. Coordinator-mediated frameworks reintroduce a single point of trust, and majority-vote frameworks over homogeneous agents fail in correlated ways; neither withstands an equivocating or adversarial member. The present approach differs by using a genuine Byzantine-agreement protocol over heterogeneous voters with the proposer excluded from voting, which is what allows it to survive the equivocation attack that defeats a naive vote. The comparison is therefore not about incremental quality on a benchmark but about a qualitative safety property that the other arrangements do not claim and were not designed to provide.

Against the automated-program-repair literature more broadly, the contribution is complementary rather than competitive. The consensus layer does not generate patches and so does not compete with search-based, semantic or learning-based generators; it governs whether their output may be applied. This means advances in any of those generation techniques can be adopted beneath the same safety boundary, which is a more durable position than tying the contribution to a particular generation method that future work may supersede.

A more concrete comparison helps locate the contribution. A single-agent repair system makes one model call to decide whether to apply a fix and offers no fault budget at all, so its safety is

undefined under a compromised or hallucinating model. A homogeneous majority of k agents drawn from one base model offers an apparent fault budget but a real one near zero against correlated error, because the agents share blind spots and can be wrong together. The consensus layer offers a defined fault budget of f within a population of $3f + 1$ heterogeneous validators, with a proven agreement property and an auditable certificate per decision. The cost is f -dependent inference and latency, but the safety property is categorical rather than probabilistic, and it is that categorical guarantee, absent from both alternatives, that justifies the additional cost for autonomous production use.

Framed as a cost-benefit trade, the comparison is straightforward for a risk-averse operator. The marginal cost of the consensus layer is the inference and latency of running $3f + 1$ validators instead of one, which is bounded and parallelisable. The marginal benefit is the elimination of an entire class of outcomes, the silent application of an unsafe or uncertified fix, that a single agent or a homogeneous majority cannot rule out. Where the expected cost of one such bad outcome exceeds the cumulative cost of the extra inference, which is the normal situation for systems handling money or customer data, the trade favours consensus decisively, and it does so on the strength of the categorical safety guarantee rather than on any contested repair-rate claim.

5.4 Critical Analysis of Findings

A critical reading has to weigh both the strength of the findings and their limits. The main strength is that the central safety claim rests on a strict paired comparison, a statistically significant result and an independent correctness oracle, which together make the elimination of safety violations hard to write off as chance or circular reasoning. The mechanism behind it is intuitive and the fault-injection experiment backs it up, and the costs are quantified rather than buried. Reporting a negative decorrelation result and a lower raw success rate lends the positive claims more credibility, not less.

Several threats to validity still have to be owned. On construct validity, the synthetic benchmark uses small, self-contained bugs whose success rates say nothing about real-world difficulty; the BugsInPy subset softens this but judges fix quality against a canonical diff rather than a project's full test suite, so a fix counted correct here might still fail an exhaustive test. On internal validity, although both pipelines share the analyzer and healer so the comparison isolates the consensus stage, the LLM providers are non-deterministic across runs and the experiments were not repeated often enough to estimate run-to-run variance, so the precise per-

scenario rates are best read as indicative. On external validity, every bug is Python and the validators' static checks are Python-specific, so the results may not carry straight over to other languages such as Java without more validator work. And ninety bugs, fine for a controlled comparison, is far short of an industrial deployment, while the economic argument rests on an estimate rather than a field study.

One more critical point concerns how general the safety result is. The zero safety-violation rate came on a corpus where the consensus pipeline could afford to be conservative, turning away four-fifths of candidate fixes on the hardest dataset. Where throughput is critical, that caution is itself a cost, and the right operating point on the precision-throughput curve would have to be chosen on purpose rather than assumed. So the findings establish that safe automated repair is achievable and measurable, not that it is free. None of this undermines the central contribution, but it marks the conditions under which it holds and sets the agenda for the future work in the next chapter.

A critical reading must confront the limitations squarely. The most significant is sample size: the repair-rate evidence rests on modest defect sets, and the headline paired comparison does not reach statistical significance. This limits the strength of any claim about how much the consensus layer improves repair outcomes, and it is the reason those outcomes are labelled preliminary throughout. The remedy is a larger replication with more defects and more provider diversity, which the reproducible harness is explicitly designed to support but which available compute did not permit within this study.

A second limitation is the oracle. The Python experiments use a run-as-script oracle that is weaker than a full test suite, so a fix counted as successful there may still be overfitted. The study mitigates this by adding the Defects4J full-suite evaluation and by reporting zero unsafe commits under that stricter standard, but the asymmetry between the strong Java oracle and the weaker Python oracle remains a threat to the interpretation of the Python repair rates. A third limitation is scope: the safety demonstration is confined to a single-host simulation and does not include the view-change that would provide liveness under a failed primary, nor multi-host deployment. These are real boundaries on what the study proves, and they are stated as such rather than glossed over.

Set against these limitations, the findings are robust where it matters most. The safety property is demonstrated by construction under the strongest single-member attack in the matrix, it does not depend on sample size because it is deterministic given the seed, and it does not depend on

the oracle because it concerns agreement rather than patch quality. A fair critical summary is therefore that the study firmly establishes the safety contribution it set out to make, while the downstream repair benefits it enables are promising but await a larger and better-instrumented evaluation.

It is useful to frame the limitations explicitly in terms of validity. Construct validity concerns whether the measures capture what they claim to; here the safety construct is well captured, because preserved agreement and zero unsafe commits are direct observations of the property of interest, whereas the repair construct is weakened on the Python set by the run-as-script oracle, which is why the Defects4J full-suite evaluation was added as a stronger measure. Internal validity concerns whether the observed differences are caused by the consensus layer rather than by confounds; the paired design, in which each defect is attempted in both modes, controls for defect difficulty, and the fixed seeds make the adversarial outcomes deterministic, which together give the internal validity its strength.

External validity concerns how far the findings generalise. The safety findings generalise well in principle, because they follow from the quorum arithmetic rather than from any property of a particular model or defect, so they should hold for other generators and other tasks within the stated fault budget. The repair findings generalise poorly at present, because they rest on small samples from a limited set of projects, and extending them is the first item of future work. Naming the limitations in these three categories rather than as a single list makes clear that the study is strong exactly where its central claim lies and weak exactly where it labels its claims preliminary, which is the alignment between confidence and evidence that the whole dissertation aims for.

CHAPTER 6: CONCLUSIONS AND RECOMMENDATIONS

6.1 Introduction

This final chapter closes the project out. It restates what the study set out to do, gives the conclusions the results support, maps them onto the objectives and research questions from Chapter 1, and offers recommendations for practice and for future research. It is kept short on purpose, since the evidence and its interpretation are already in Chapters 4 and 5; the job here is to consolidate, not to introduce.

This final chapter draws the study to a close by stating its conclusions in proportion to the evidence, offering recommendations for practice and for future research, and reflecting on the broader idea the work contributes. The conclusions distinguish the firm safety result from the preliminary repair result, the recommendations translate the findings into actionable guidance, and the reflection identifies the durable methodological contribution that future work can build upon.

6.2 Conclusions

The project set out to design, build and empirically validate a Byzantine fault-tolerant multi-agent system for safe automated program repair in e-commerce systems, and it did. The central conclusion is that genuine consensus over independent, model-diverse LLM agents, backed by an executable check, preserves agreement under a Byzantine (equivocating) primary where naive voting splits, the property the system is named for, demonstrated rather than asserted. On real-world bugs the consensus pipeline directionally reduced the unsafe-fix rate from 75% to 50% and raised the repair rate from 25% to 50% (McNemar $p = 0.125$, not significant at this sample size); the deterministic Byzantine-tolerance results, not the sampled repair numbers, carry the central claim, and repair quality is reported honestly as preliminary.

A second conclusion is that the safety guarantee comes from defence in depth, not consensus alone. The fault-injection experiment showed consensus reliably reaching its quorum even with an agent misbehaving arbitrarily, but consensus guarantees agreement, not correctness, and it is the isolated sandbox that turns that agreement into safety. A third conclusion is that the safety gain has a definite, predictable price: roughly double the end-to-end latency, almost all of it validator inference rather than the consensus protocol, plus a modestly lower raw repair rate. A fourth, reported honestly as a negative result, is that on clear-cut bugs the diversity of the

agent population did not produce the decorrelated failures the design was built to exploit; showing that benefit would take borderline bugs on which competent models genuinely disagree. Table 6.1 maps these conclusions onto the project's five objectives.

Table 6.1. *Mapping of project objectives to delivered work.*

Objective	Realisation in this project	Status
O1: multi-agent architecture with PBFT consensus	Analyzer, healer and validator agents implemented, each on a different LLM provider; three-phase PBFT engine built.	Met
O2: sandbox and e-commerce invariants	Isolated Docker sandbox with subprocess fallback; payment, inventory, refund and currency invariants embedded in the validation.	Met
O3: empirical evaluation against a baseline	Ninety paired bugs across three datasets; consensus tolerated one Byzantine validator across five fault behaviours, an $f = 2$ study showed the bound is tight, and an independent-replica realisation preserved agreement under an equivocating primary where naive voting split (Sections 4.4.5 to 4.4.8).	Met
O4: decorrelation measurement and fault injection	Pairwise validator agreement measured; five Byzantine behaviours injected with consensus success at 100%.	Met
O5: open-source release	Prototype, evaluation harness and datasets released for reproduction and extension.	Met

Against the research questions, the study answers all four. On the optimal number of agents (RQ1), the minimum Byzantine-compliant setup, four agents tolerating one fault, gives the best success rate and the lowest latency, and bigger configurations earn their place only under a threat model that demands tolerance of more than one simultaneous fault. On consensus latency

(RQ2), the protocol itself is negligible at under two milliseconds a round, and the observed increase is the cost of extra validator inference, defensible for high-stakes commits. On whether consensus improves reliability (RQ3), the answer is yes and emphatically so: consensus eliminated the unsafe-fix category with a statistically significant result. On the economic benefit (RQ4), a conservative estimate from the CISQ (2024) downtime figures says that avoiding unsafe fixes in high-stakes production repairs averts far more loss than the modest cloud-inference cost of consensus, so the approach pays wherever a repair carries financial risk.

The study set out to determine whether genuine Byzantine fault tolerance can be brought to bear on the decision to apply an automated repair, and whether doing so yields a demonstrable safety guarantee. The answer, on the evidence gathered, is that it can and it does. A consensus layer of $3f + 1$ model-diverse validators with a $2f + 1$ quorum, excluding the proposer from voting and authenticating every message, preserves agreement under an equivocating primary where a naive vote splits, rejects forged messages, tolerates faults up to the replication bound and fails safely beyond it. This is the core conclusion, and it earns the project its title on the safety axis through direct demonstration rather than assertion.

The secondary conclusions are reported with calibrated confidence. The consensus layer is associated with improved safety-violation and repair rates on the real-world subset, in a consistent direction, but the effect does not reach statistical significance on the available sample and is therefore preliminary. Model diversity reduces correlated error without eliminating it, and the e-commerce invariant gate acts as defence in depth rather than as a primary filter. Each of these is stated as the data warrant, and together they describe an approach whose safety contribution is firm and whose repair contribution is promising and worth scaling.

The broader conclusion is methodological. By treating an unreliable LLM agent as a Byzantine replica and applying a protocol with a proven safety property, the study shows that the decades of distributed-systems theory developed for crashed and compromised servers can be repurposed to govern stochastic reasoning agents. This reframing is the durable idea the dissertation contributes, and it is independent of any particular model or benchmark: as generators improve, the same safety boundary continues to apply, which is what makes the contribution worth building on.

Stated as a single contribution, the dissertation shows that a stochastic reasoning agent can be governed by the same Byzantine-agreement machinery developed for crashed and compromised servers, and that doing so yields a safety guarantee that is demonstrated rather

than asserted. The firm result is the preservation of agreement under an equivocating primary within a stated fault budget; the preliminary results are the directional improvements in repair and safety-violation rates; and the negative results on decorrelation and domain invariants locate the boundaries of what diversity and invariant checking add. The value of the work is that each of these is reported in proportion to its evidence, so that what a reader takes away is calibrated to what the study actually established.

Finally, the contribution is durable because it is decoupled from the rapid churn of model capability. The safety property derives from the protocol, not from any model, so as generators improve the same boundary continues to apply and the same guarantee continues to hold; better models simply produce more candidates that clear the quorum. A study tied to a particular model's repair performance would age quickly, whereas a study that establishes a protocol-level guarantee provides a foundation that later, more capable systems can be built upon without revisiting the safety argument. That durability is the reason the dissertation frames its central result in terms of agreement under faults rather than in terms of any benchmark score.

6.3 Recommendations

On the basis of these conclusions, the following recommendations are offered, first for practice and then for future research.

6.3.1 Recommendations for Practice

- **Adopt consensus for high-stakes repair:** organisations deploying LLM-driven automated repair to high-stakes production code, particularly payment, settlement and inventory logic, should adopt a consensus-plus-sandbox architecture rather than a single-agent pipeline, accepting the additional latency in exchange for the elimination of unsafe fixes.
- **Always retain an independent oracle:** because consensus guarantees agreement and not correctness, an independent execution check with domain-specific invariants should always be the final gate before a fix is committed.
- **Use the minimum compliant configuration:** the four-agent, single-fault configuration is recommended as the default; larger populations should be used only where the threat model explicitly requires tolerance of multiple simultaneous faults.

- **Accelerate validators, not the protocol:** because validator inference dominates latency, practitioners should invest in faster or hardware-accelerated validator models rather than in optimising the consensus protocol.

For practitioners considering autonomous remediation, the central recommendation is to treat the safety boundary as fixed and auditable and the generator as replaceable. An operator should require that no fix reaches production without clearing an independent, model-diverse quorum and a sandbox that enforces domain invariants, and should accept the additional inference latency as the price of a guarantee that a single agent cannot provide. The consensus layer should be configured with a fault budget appropriate to the deployment, recognising that the protocol fails safely beyond that budget, and the audit trail of messages, votes and certificates should be retained so that any committed decision can be reconstructed and justified after the fact.

6.3.2 Recommendations for Future Research

- **Develop a semantic-equivalence quorum:** the principal theoretical extension is to define a probabilistic consensus relation parameterised by an application-supplied semantic-equivalence function, with bounds on agreement probability as a function of agent correctness and ensemble size, so that consensus can be reached on semantically equivalent rather than byte-identical fixes.
- **Test decorrelation on borderline bugs:** the failure-decorrelation hypothesis should be tested on a purpose-built dataset of borderline bugs on which competent models genuinely disagree, since the present clear-cut bugs did not exercise it.
- **Extend to other languages and scale:** the evaluation should be extended to Java using Defects4J, which requires building language-specific validator checks, and to a larger corpus approaching industrial scale.
- **Measure recovery under continuous faults:** faults should be injected at random points during a continuous multi-round repair stream so that recovery time, from fault detection to restored throughput, can be measured.
- **Conduct a live deployment study:** a controlled multi-month deployment on a real e-commerce platform should be conducted to replace the estimated economic argument with measured evidence.

To close, the project has shown that a decades-old discipline, Byzantine fault tolerance, can be carried productively into a brand-new setting, multi-agent LLM program repair, and that doing so makes automated repair safe enough to consider in places where a single wrong fix is unacceptable. The prototype, the benchmark and the evidence are released so that others can build on them.

Three lines of future work follow directly from the limitations identified. The first is a large-scale replication of the repair-rate evaluation, with a substantially larger defect set and more provider diversity, so that the directional improvement observed here can be confirmed or refuted with adequate statistical power. The reproducible harness was built with this extension in mind, and the main barrier is compute rather than design, so this is the most immediately actionable next step.

The second line is to complete the protocol on the liveness axis. Implementing the PBFT view-change would allow the system to make progress under a failed or permanently-faulty primary rather than merely failing safely, and deploying the validators across multiple physical hosts would move the demonstration from a single-host simulation toward a realistic distributed setting. Together these would let a future study claim both safety and liveness under the full PBFT fault model. The third line is to strengthen the oracle throughout, replacing the run-as-script Python oracle with full-suite verification comparable to the Defects4J evaluation, so that reported repair rates carry the same confidence as the safety results. Pursuing these three directions would convert the present study's firm but bounded safety contribution into a complete and well-powered account of Byzantine fault-tolerant autonomous repair.

A fourth and more exploratory direction would investigate reputation-weighted voting and the knowledge-graph layer that are implemented in the codebase but not evaluated in this study. Weighting a validator's vote by its historical reliability could in principle improve the quality of accepted fixes without weakening the structural safety guarantee, provided the weighting never lets a single member reach a quorum alone. Evaluating whether such weighting helps, and whether it introduces new correlated-failure risks, is a natural extension once the larger replication has established a stable baseline against which any improvement can be measured.

REFERENCES

- Abraham, I., Dolev, D., & Halpern, J. Y. (2008). An almost-surely terminating polynomial protocol for asynchronous Byzantine agreement with optimal resilience. In *Proceedings of the 27th ACM Symposium on Principles of Distributed Computing (PODC)* (pp. 258-267). ACM.
- Aublin, P.-L., Mokhtar, S. B., & Quema, V. (2013). RBFT: Redundant Byzantine fault tolerance. In *Proceedings of the 33rd IEEE International Conference on Distributed Computing Systems (ICDCS)* (pp. 297-306). IEEE.
- Berabi, B., He, J., Raychev, V., & Vechev, M. (2021). TFix: Learning to fix coding errors with a text-to-text transformer. In *Proceedings of the 38th International Conference on Machine Learning (ICML)* (Vol. 139, pp. 780-791). PMLR.
- Bessani, A., Sousa, J., & Alchieri, E. (2014). State machine replication for the masses with BFT-SMaRt. In *Proceedings of the 44th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)* (pp. 355-362). IEEE.
- Blanchard, P., El Mhamdi, E. M., Guerraoui, R., & Stainer, J. (2018). Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)* 30 (pp. 119-129). Curran Associates.
- Buchman, E. (2016). Tendermint: Byzantine fault tolerance in the age of blockchains [Master's thesis, University of Guelph].
- Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. In *Proceedings of the 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (pp. 173-186). USENIX Association.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. (2021). Evaluating large language models trained on code (arXiv:2107.03374).
- Clement, A., Wong, E., Alvisi, L., Dahlin, M., & Marchetti, M. (2009). Making Byzantine fault tolerant systems tolerate Byzantine faults. In *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI)* (pp. 153-168). USENIX Association.
- Consortium for Information and Software Quality (CISQ). (2024). The cost of poor software quality in the United States: A 2022 report (updated 2024). CISQ.

- Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2024). Improving factuality and reasoning in language models through multi-agent debate. In Proceedings of the 41st International Conference on Machine Learning (ICML).
- Dwork, C., Lynch, N., & Stockmeyer, L. (1988). Consensus in the presence of partial synchrony. *Journal of the ACM*, 35(2), 288-323.
- Fischer, M. J., Lynch, N. A., & Paterson, M. S. (1985). Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2), 374-382.
- Gueta, G. G., Abraham, I., Grossman, S., et al. (2019). SBFT: A scalable and decentralized trust infrastructure. In Proceedings of the 49th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN) (pp. 568-580). IEEE.
- Gupta, R., Pal, S., Kanade, A., & Shevade, S. (2017). DeepFix: Fixing common C language errors by deep learning. In Proceedings of the 31st AAAI Conference on Artificial Intelligence (pp. 1345-1351). AAAI Press.
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In Proceedings of the 12th International Conference on Learning Representations (ICLR).
- Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In Proceedings of the International Symposium on Software Testing and Analysis (ISSTA) (pp. 437-440). ACM.
- Kotla, R., Alvisi, L., Dahlin, M., Clement, A., & Wong, E. (2007). Zyzzyva: Speculative Byzantine fault tolerance. In Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP) (pp. 45-58). ACM.
- Lamport, L., Shostak, R., & Pease, M. (1982). The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3), 382-401.
- Le Goues, C., Nguyen, T., Forrest, S., & Weimer, W. (2012). GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1), 54-72.
- Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., & Ghanem, B. (2023). CAMEL: Communicative agents for 'mind' exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS) 36*. Curran Associates.

- Marginean, A., Bader, J., Chandra, S., Harman, M., Jia, Y., Mao, K., et al. (2019). SapFix: Automated end-to-end repair at scale. In Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP) (pp. 269-278). IEEE.
- Mechtaev, S., Yi, J., & Roychoudhury, A. (2016). Angelix: Scalable multiline program patch synthesis via symbolic analysis. In Proceedings of the 38th International Conference on Software Engineering (ICSE) (pp. 691-701). ACM.
- Miller, A., Xia, Y., Croman, K., Shi, E., & Song, D. (2016). The honey badger of BFT protocols. In Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS) (pp. 31-42). ACM.
- Nguyen, H. D. T., Qi, D., Roychoudhury, A., & Chandra, S. (2013). SemFix: Program repair via semantic analysis. In Proceedings of the 35th International Conference on Software Engineering (ICSE) (pp. 772-781). IEEE.
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., et al. (2024). ChatDev: Communicative agents for software development. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL). ACL.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems (NeurIPS) 36.
- Smith, E. K., Barr, E. T., Le Goues, C., & Brun, Y. (2015). Is the cure worse than the disease? Overfitting in automated program repair. In Proceedings of the 10th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE) (pp. 532-543). ACM.
- Veronese, G. S., Correia, M., Bessani, A. N., Lung, L. C., & Verissimo, P. (2013). Efficient Byzantine fault-tolerance. IEEE Transactions on Computers, 62(1), 16-30.
- Widyasari, R., Sim, S. Q., Lok, C., Qi, H., Phan, J., Tay, Q., et al. (2020). BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE) (pp. 1556-1560). ACM.

- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., et al. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In ICLR 2024 Workshop on Large Language Model (LLM) Agents.
- Xia, C. S., & Zhang, L. (2022). Less training, more repairing please: Revisiting automated program repair via zero-shot learning. In Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE) (pp. 959-971). ACM.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In Proceedings of the 11th International Conference on Learning Representations (ICLR).
- Yin, M., Malkhi, D., Reiter, M. K., Gueta, G. G., & Abraham, I. (2019). HotStuff: BFT consensus with linearity and responsiveness. In Proceedings of the 38th ACM Symposium on Principles of Distributed Computing (PODC) (pp. 347-356). ACM.
- Dietterich, T. G. (2000). Ensemble methods in machine learning. In Multiple Classifier Systems, LNCS 1857 (pp. 1-15). Springer.
- Maturi, M. H., De La Cruz, E., Addula, S. R., et al. (2025). Enhancing smart contract security with explainable AI: A framework for re-entrancy vulnerability detection and explanation. In 2025 IEEE SIEDS. IEEE.
- Wang, X., Wei, J., Schuurmans, D., et al. (2023). Self-consistency improves chain-of-thought reasoning in language models. In ICLR.